CHAPTER 1

INTRODUCTION

1.1 PROJET DESCRIPTION

The Organic Farming System is a digital platform designed to help farmers, customers, and administrators promote sustainable agriculture. Organic farming has become crucial today as people favor healthier food choices and eco-friendly practices. However, managing organic product sales, ensuring quality, and maintaining trust between farmers and consumers can be difficult when done manually.

This system offers a centralized solution where farmers can register and list their organic products. Customers can directly view, purchase, and support local farming communities. Unlike traditional farming management, the platform promotes transparency by displaying product details, certifications, and fair pricing.

The backend features a secure database and authentication system that ensures only verified farmers and users can participate. It integrates payments with trusted gateways for safe transactions. Meanwhile, the frontend provides a user-friendly interface for seamless interaction among farmers, admins, and customers.

An admin dashboard helps monitor activities like product listings, user management, payments, and handling complaints. By digitizing the organic farming process, the system reduces paperwork, lowers the risk of fraud, and strengthens trust between producers and consumers.

The system is scalable and can be expanded with features like community support, real-time communication between farmers and consumers, and IoT-based crop monitoring in the future. The project provides not only as a management tool but also as a step toward smarter and more sustainable agriculture.

1.2 PROBLEM STATEMENT

Organic farming is becoming popular because people care about health, the environment, and sustainability. In this project their is problems in connecting farmers and customers. Farmers do not always have proper digital tools to show their products, and they often depend on

middlemen or local markets, which lowers their profits. They also face difficulties in keeping records of sales and payments, which can lead to mistakes. Customers, on the other hand, find it hard to get verified organic products, compare prices, or be sure about the quality. Because of this, trust becomes a big issue.

Most of the current methods are offline, which makes the process slow, costly, and less transparent. Without a proper system, administrators also struggle to monitor sales, handle complaints, and ensure smooth operations. To solve these issues, a web-based Organic Farming System is needed. This system will let farmers register and list their products. Customers can buy genuine items through secure payments. Administrators can manage and oversee everything in one place. It will make the process fair, trustworthy, and efficient, helping both farmers and customers while promoting sustainable farming.

1.3 OBJECTIVES

- The system aims to provide a centralized digital marketplace that connects administrators, farmers, and customers in a structured and reliable way. Role-based access ensures that each stakeholder—farmer, customer, or admin—interacts only with the features relevant to them. Secure login and verification mechanisms help maintain data privacy and build trust, creating a safe environment for transactions and interactions.
- Farmers can easily register and showcase their organic products with essential details like price, quantity, and certification. Customers can then browse, compare, and purchase directly from verified farmers. Once an order is placed, customers can track its status step by step, such as order received, packed, shipped, and delivered, ensuring full transparency. A built-in feedback and complaint module further strengthens accountability by allowing customers to share their experiences, which admins can review and resolve.
- Beyond enabling product sales, the platform also promotes community participation and sustainable farming practices. Features like complaint resolution, customer feedback, and transparent pricing empower both farmers and buyers. Administrators can organize awareness campaigns, promote certified products, and support farmers in showcasing authentic organic produce, thereby strengthening trust and credibility.
- To improve decision-making and efficiency, the system integrates reporting and monitoring tools for administrators. These offer insights into sales trends, demand distribution, and farmer participation. They assist with planning and support. The platform is designed with scalability in mind, meaning it can grow to serve larger farming communities, support advanced features like IoT-based crop tracking, or expand to cover broader agricultural needs. This ensures the system remains relevant, flexible, and impactful in the long run.
-

1.4 COMPANY PROFILE

ORGANIZATION STRUCTURE

Zephyr Technologies, located in Mangalore, is a reliable company that provides software development services and practical training for computer science and IT students. It offers solid support for academic projects, internships, and skill development aimed at careers. The company trains students in popular technologies, including Python, Machine Learning, Artificial Intelligence, Android Development, Web Technologies, IoT, and more. One of its key strengths is helping students work on real-time projects, giving them practical experience and exposure to how software is developed in the industry. Experienced trainers guide students step-by-step, making learning easy to understand even for beginners. Zephyr Technologies also offers a good learning environment with air-conditioned classrooms, Wi-Fi-enabled labs, smartboards, and well-maintained infrastructure. It focuses not only on technical skills but also on project explanation, documentation, and presentation, helping students confidently face academic reviews and job interviews. Known for its student-friendly approach and updated training methods, Zephyr Technologies continues to support learners in becoming job-ready and confident in the software field.

VISION

Zephyr Technologies, based in Mangalore, has a clear vision to be a place where students, recent graduates, and young professionals can grow into confident software developers through proper guidance and hands-on learning. The company believes that anyone can learn software development when they receive the right guidance. This includes step-by-step support, hands-on experience, and encouragement.

The long-term objective is to establish a reliable learning environment where students are comfortable posing queries, experimenting with new tools, and developing actual projects. Zephyr wants to give students the experience they need for interviews, real-world project scenarios, and workplace challenges in addition to the technical knowledge they need. Helping students understand that learning by doing is the best way to develop is part of the vision. They can gain confidence in their field and become prepared for the workforce by doing this.

MISSION

The mission of Zephyr Technologies is to support students at every stage of their learning journey, especially during their academic projects, internships, and skill development. The company focuses on making tough topics easier to understand by using simple explanations, real-life examples, and hands-on sessions. Beyond classroom theory, the objective is to assist students in applying what they have learnt to real-world software tasks.

Zephyr believes that personalized guidance, friendly trainers, and clear learning paths can greatly impact how students learn The company's goal is to develop confidence, problem-solving abilities, and the capacity to collaborate with actual software teams in addition to teaching tools and technologies. Its goal is to help every student feel ready for whatever comes next, be it a job, more education, or beginning a project.

SERVICES OFFERED BY THE ZEPHYR TECHNOLOGIES

Zephyr Technologies supports students in the computer and IT fields by providing practical help during their studies. One of their key services is guiding students through final-year projects. Whether it's choosing a topic, writing code, troubleshooting problems, or preparing a report, the team offers comprehensive assistance. Their goal is for every student to comprehend their project and be able to confidently explain it in a review or presentation.

In addition to project support, Zephyr offers internships and short courses in various in-demand technologies. Python programming, Android development, IoT, basic machine learning, and HTML, CSS, and JavaScript website development are a few examples. In addition to listening in lectures, students also practise using real-world examples. They occasionally even get to watch or assist with projects for real clients, which gives them a better understanding of how software is created in the real world.

COMPANY PROCESS AND CULTURE

At Zephyr Technologies, the learning process is well-organized and easy to follow. When a student joins, the trainers first assess their current knowledge and experience. Based on that, they create a plan, starting with basic concepts and gradually guiding them through development, testing, and final presentation. Students receive daily tasks and regular feedback to help them stay on track and improve step by step.

The company culture is very welcoming and focused on student growth. Trainers are friendly, patient, and willing to answer questions, no matter how small. Students never feel nervous or rushed. The classrooms are clean, air-conditioned, and equipped with Wi-Fi. The labs have everything needed for software training. Students can freely ask questions, collaborate in groups, and learn by doing in Zephyr's peaceful, encouraging environment. The environment encourages confidence, discipline, curiosity, and respect—all of which are critical for success in the software industry.

CHAPTER 2

LITERATURE SURVEY

Patil et al. (2016) proposed a computerized system for managing agriculture that enabled farmers to document information about yields, soil conditions, and crops. Their study showed how digital platforms could reduce manual errors, improve record-keeping, and create a structured way to track farming activities. While the system had a simple design, it laid the groundwork for later projects aimed at developing farmer-focused platforms to boost productivity.

Verma and Singh (2018) introduced an e-commerce approach to connect farmers directly with customers. Their research aimed to eliminate intermediaries, which would increase profits for farmers and ensure fair prices for consumers. The study highlighted the importance of trust and transparency in agricultural transactions, inspiring future systems to include verification processes and complaint management features.

Rathod et al. (2019) developed a web-based crop management and advisory system. Their platform offered real-time recommendations on crop selection, pest control, and fertilizer use. The research illustrated how adding decision-support tools to farming applications could help farmers enhance yield quality while also promoting environmental sustainability. This work demonstrated that beyond sales, digital platforms can serve as knowledge hubs for farmers.

Kumar et al. (2020) explored the design of online agricultural marketplaces. Their system included features like product listing, order management, and online payment options. The study demonstrated how farmers could use web technologies to reach markets outside of their local area. They did, however, also highlight issues with user adoption brought on by rural farmers' lack of digital literacy and training.

Sharma et al. (2022) proposed an integrated organic farming portal containing modules for farmer registration, product verification, and supply chain tracking. Their study emphasised the need for trust-building systems and the growing consumer demand for organic products. By including certification checks and complaint resolution features, their platform aimed to ensure product authenticity and strengthen relationships between farmers and consumers.

Gupta et al. (2023) focused on improving transparency in agricultural e-commerce by incorporating blockchain technology. Their research showed how decentralized ledgers could

guarantee trust in the authenticity of organic products, secure transactions, and fair pricing. This strategy demonstrated how new technologies have the ability to influence how sustainable agriculture platforms develop in the future.

2.1 EXISTING SYSTEM AND PROPOSED SYSTEM

2.1.1 EXISTING SYSTEM

Most current systems sell organic products through regular e-commerce platforms that are not tailored for farmers. These platforms sell various goods, making it hard for customers to verify whether a product is genuinely organic. Farmers struggle because they frequently depend on middlemen to reach buyers and have few options for showcasing the quality of their goods.

As a result, farmers do not receive fair prices, and buyers sometimes pay more for products that may lack certification. Technically, these platforms are basic product listing sites, lacking emphasis on organic certification, farmer-customer interaction, or building trust. Consequently, existing systems fall short in meeting the unique needs of organic farming.

2.1.2 PROPOSED SYSTEM

The proposed Organic Farming System is designed to address these issues by creating a dedicated platform for farmers, customers, and administrators. Farmers can register, list their products, and upload certificates to verify that their produce is organic. Customers can easily browse and purchase products with confidence, knowing everything is verified by the admin. This builds trust and reduces the reliance on middlemen, allowing farmers to earn fair wages while customers pay reasonable prices.

From a technical perspective, the system features an intuitive frontend for easy navigation and a secure backend to manage user data, authentication, and product information. Each role—farmer, admin, customer—has defined permissions, ensuring the system is organized and secure. This system not only connects farmers directly with consumers but also fosters transparency and encourages healthy living through certified organic products.

2.2 FEASIBILITY STUDY

Before building the Organic Farming System, a feasibility study assessed whether the idea was practical, useful, and feasible with the available resources. The study examined the technical aspects, ease of use in real-life scenarios, and overall development costs.

One challenge was managing different roles—farmers, customers, and admins—while ensuring security and effective communication. Another issue involved creating a system that could verify organic products and maintain trustworthy records. However, the study confirmed that with modern web technologies and open-source tools, developing such a platform is possible and cost-effective.

2.2.1 TECHNICAL FEASIBILITY

The system's technical feasibility is strong as it relies on widely available and dependable technologies. The frontend is built with React (or HTML/CSS/JavaScript based on your project) to provide users with a clean and simple interface. The backend utilizes Node.js with Express and MongoDB for secure data management. These technologies are open-source and well-supported, which makes development easier and more affordable.

Since the system is web-based, it can operate on standard computers and mobile devices with internet access. There is no need for expensive hardware or software, making the project technically feasible.

2.2.2 OPERATIONAL FEASIBILITY

Operationally, the system is very practical. Farmers can register and upload products, customers can browse and buy items, and admins can manage product verification and user activities. The platform is designed to be user-friendly, allowing even those with limited technical skills to navigate it easily. This web-based structure ensures that anyone with a browser can access the system at any time and place. This adaptability makes it suitable for farmers in rural areas, consumers in cities, and administrators monitoring the platform. Overall, the system is straightforward to operate and aligns with its intended purpose.

2.2.3 ECONOMIC FEASIBILITY

The system is economically viable since it is built using free and open-source tools like React, Node.js, Express, and MongoDB. There are no costly licenses or subscription fees involved. Development and hosting can take place on affordable servers, and cloud platforms offer low-cost scaling options if needed. Farmers and customers do not need special equipment—just a mobile phone or computer with internet access. This affordability supports sustainable long-term use. Future updates can also be carried out at low cost, making the system financially practical for both small and large-scale adoption.

2.3 TOOLS AND TECHNOLOGIES USED

2.3.1 FRONT-END:

1.REACT.JS

React.js serves as the backbone of the frontend, providing a component-based structure that ensures modularity, reusability, and scalability. Each part of the Organic Farming System, such as farmer registration, product catalog, shopping cart, and complaint management, is developed as a separate React component, keeping the code organized and maintainable. The use of React Hooks and state management libraries, like Redux or Context API, allows dynamic updates, such as product availability or live order tracking, without needing to reload the page. React's virtual DOM ensures quick rendering, enhancing user experience for both farmers managing their listings and customers browsing products. Its extensive ecosystem makes integrating third-party libraries for features like data visualization, notifications, or interactive forms easier..

2.TAILWINDCSS

Tailwind CSS builds a modern, responsive, and consistent user interface through utility-first classes. It maintains uniform design across the platform, keeping colors, typography, and spacing consistent. Tailwind's responsive grid system adjusts layouts for desktops, tablets, and smartphones, making the platform usable for all participants. Farmers can upload products using a mobile device, while customers can browse and shop comfortably on any screen. Tailwind also offers strong customization options, enabling the system to incorporate branding and reusable style patterns, which speeds up development and reduces repetitive coding.

3.REAL-TIME LIBRARIES (CHAT & MAP INTEGRATION)

The frontend integrates real-time libraries to boost user interaction. A built-in chat feature enables direct communication between customers, farmers, and administrators for inquiries or complaints. Additionally, real-time notifications keep customers informed about order statuses, such as confirmed, shipped, or delivered, while farmers receive alerts about new purchases or low stock. This enhances communication, builds trust, and improves the shopping experience for all users.

2.3.2 BACK-END:

1. NODE.JS

Node.js drives the backend with its event-driven and non-blocking architecture, allowing efficient management of multiple requests simultaneously. For example, the system can handle a purchase, record a complaint, and process a new product listing all at once without any delays. Its

asynchronous nature supports smooth operations, even as the platform scales to accommodate a larger farming community. With a wide range of open-source modules, Node.js simplifies integration of features like authentication, payment gateways, and data validation.

2. EXPRESS.JS

Express.js serves as the main framework for building RESTful APIs that manage all interactions between the frontend and backend. This encompasses customer login, order placement, product searches, and complaint submissions. Middleware functions validate inputs, ensuring proper product details or secure payment information and handling errors gracefully to maintain system stability. Express also organizes backend logic into structured routes and controllers, making maintenance and expansion easier when adding new features like discount coupons or subscription purchases..

3. MONGODB

MongoDB is selected as the primary database for its flexibility in managing various data types, such as farmer profiles, product listings, customer orders, and complaint records. Its document-oriented model supports quick and efficient data operations, reflecting new products or orders instantly on the platform. MongoDB's scalability through replication and sharding ensures that the system can grow to accommodate more farmers and customers in the future without performance issues.

4. JWT (JSON WEB TOKENS)

JWT enables secure and role-based authentication within the system. When a user logs in, a token is created that encodes their role (Admin, Farmer, or Customer). This ensures each role has access only to permitted actions, such as farmers listing products, customers making purchases, and admins managing operations. By including expiration times, JWT enhances security and reduces the risk of unauthorized access.

5. NODEMAILER

Nodemailer manages automated email communications. It sends confirmation emails to customers after purchases, alerts farmers about new orders, and notifies admins about complaints or system updates. Customizable email templates provide professional, branded communication, helping to foster trust and engagement among stakeholders.

6. WEBSOCKET (REAL-TIME COMMUNICATION)

WebSocket enables instantaneous updates across the system. Instead of refreshing their page, customers receive real-time updates on order status changes, such as shipped or delivered. Similarly, farmers get live notifications when a new purchase occurs. This low-latency communication boosts efficiency, customer satisfaction, and transparency throughout the platform.

7. **PAYMENT GATEWAY INTEGRATION**

To manage secure online transactions, the backend includes payment gateway APIs. This allows customers to make smooth digital payments using credit cards, UPI, or net banking, while ensuring encryption and compliance with financial security policies. Payment logs are safely stored in MongoDB for future reference and auditing, adding an extra layer of trust to the platform.

CHAPTER 3

REQUIREMENT SPECIFICATION

3.1 PURPOSE

The goal of this project is to create an Organic Farming System that connects farmers and customers through a single digital platform. This system allows farmers to easily showcase and sell their organic produce directly to consumers. Customers can enjoy a reliable marketplace for healthy, chemical-free products.

Instead of relying on middlemen or selling only locally, this system enables farmers to reach a broader audience. They are able to control their product listings, track orders, and receive digital payments. Customers can register, browse organic goods, place orders securely, and download receipts.

By incorporating features like farmer-managed product catalogs, secure customer registration with OTP verification, order booking, online payment integration, and admin monitoring, this project aims to improve fair trade, transparency, and accessibility in organic farming. Ultimately, the system supports sustainable agriculture while offering a convenient and scalable platform for both farmers and consumers.

3.2 SCOPE

This project aims to create an interactive web platform that fosters direct collaboration between farmers and customers in organic farming.

- **For Farmers:** The system lets farmers register, log in, add or manage organic products, update stock availability, and track sales.
- **For Customers:** The system enables secure registration with OTP verification, browsing available organic products, placing orders with online payments, and tracking order history with downloadable receipts.
- **For Admins:** It provides a centralized dashboard to monitor users, products, and sales analytics, ensuring the system runs smoothly.

The platform also offers extra features such as search and filtering options for customers, PDF receipt generation for purchase records, and charts for admins to track system performance. By integrating these features, the system provides a transparent and user-friendly solution that encourages organic farming while streamlining buying and selling.

3.3 FUNCTIONAL REQUIREMENTS

The Organic Farming System helps farmers, customers, and admins do their work in an easy and safe way. Farmers can create an account, log in, and add their organic products. They can also update product details, eliminate anything that isn't available, and check customer orders to update delivery status.

Customers can register safely using OTP or email, then log in to browse products, use filters to search, place orders, and pay online. They also get PDF receipts to keep records of what they bought. Admins manage the whole system. They receive reports to monitor performance and can see the number of users, orders, and sales. They make sure all product and order details are correct. The system stores all data securely, updates in real time, and shows clear messages when actions are successful or fail.

3.4 NON-FUNCTIONAL REQUIREMENTS

The Organic Farming System is designed to give a smooth and reliable experience for everyone using it. It is capable of supporting numerous users simultaneously without experiencing any lag. even during busy hours. Security is a top priority, with features like OTP verification, secure login using JWT, and encrypted payments to keep personal and financial details safe.

The system is also built to grow in the future. Since it is developed with the MERN stack, it can easily support more users, more products, and extra features without affecting performance. The design is simple, responsive, and works well on all devices such as computers, tablets, and mobile phones. Reliability is another key focus. The system ensures orders are processed correctly, handles errors smoothly if users enter wrong information or if a process fails, and includes backup options to protect important data. Users can always rely on the system to function correctly in this way.

3.5 HARDWARE AND SOFTWARE REQUIREMENTS

3.5.1 HARDWARE REQUIREMENTS

- Operating System: Windows 10 / Windows 11 / Linux (Ubuntu recommended for deployment)
- Processor: Quad-core processor (Intel i5 or above / AMD Ryzen equivalent)
- Hard Disk: Minimum 40 GB free space
- RAM: 8 GB (16 GB recommended for smooth operation)
- Input Devices: Standard Keyboard and Mouse
- Network:Dependable internet access for online transactions, API calls, and cloud hosting

3.5.2 SOFTWARE REQUIREMENTS

- Programming Languages: JavaScript (ES6+), Node.js (v16 or above)
- Frontend Framework: React.js with CSS for responsive UI
- Backend Framework: Express.js (Node.js)
- Database: MongoDB (local or cloud via MongoDB Atlas)
- Package Manager: npm (Node Package Manager)
- Development Tools: Visual Studio Code with debugging/linting extensions
- Additional Tools: PDF generation library (for receipts), Chart libraries (for admin analytics)
- Browser: Google Chrome / Microsoft Edge / Mozilla Firefox

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM DESIGN

4.1.1 INTRODUCTION

System design is essential to the Organic Farming System. It outlines how the platform's architecture, modules, and data flow work together to meet the project's functional and non-functional needs. The design promotes smooth interactions between farmers, customers, and administrators. It focuses on secure registration, product management, online ordering, and payment processing.

The system adopts a modular approach. Each main feature, such as farmer product management, customer order handling, payment integration, and admin analytics, operates as an independent module connected to the central system. This modularity improves maintenance, scalability, and future growth. For example, new modules like AI-based crop demand prediction or mobile app integration can be added without disrupting the current setup.

Particular attention is given to real-time operations, like updating product availability, generating order receipts instantly, and keeping admin dashboards current. Security, reliability, and data integrity form the design's foundation. These components guarantee that private data, such as login credentials, payment information, and transaction history stay secure while the platform delivers uninterrupted service, even during high demand.

4.1.2 SCOPE

The Organic Farming System is meant to bring farmers and customers together, while administrators make sure everything works smoothly. Farmers can create an account, log in, and manage their products by adding, updating, or removing them. They can also see customer orders and update delivery status, which keeps buyers informed.

For customers, the system is easy to use. They can search for organic products, filter by category or price, add items to their cart, place orders, and pay securely online. They also get digital receipts in PDF format and can check their past orders, which makes the process simple and trustworthy. Admins take care of the whole platform. They can track users, monitor sales,

and create reports to understand system performance. Additionally, they verify that all order and product details are accurate. The system includes secure payments, PDF receipts, and real-time updates using MongoDB. With safety features like OTP/email verification and JWT login, it creates a reliable and safe marketplace for organic products.

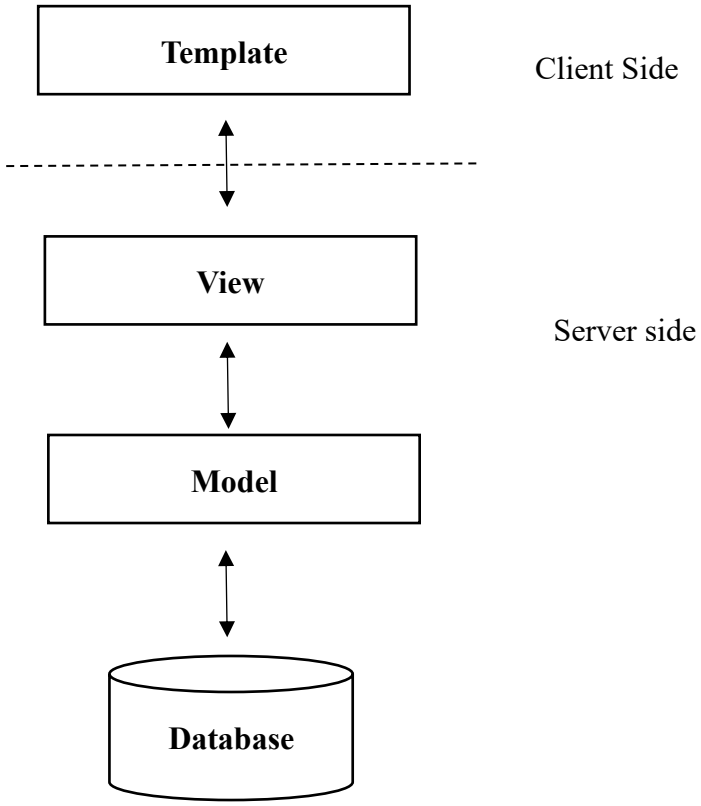
4.1.3 AUDIENCE

The Organic Farming System is made for a variety of people who are essential to its operation. Administrators use the system as a control center. They manage users, check orders, keep the platform secure, and make sure all data is correct. They also look at reports to make better decisions. Farmers are the main suppliers. They add their organic products, update details, and manage customer orders. By doing this, they deliver fresh and genuine products directly to customers without middlemen, which also supports sustainable farming.

Customers are the main users who benefit the most. They can browse products, place orders, pay safely, and track their purchases with order history and downloadable receipts. The system is made simple so even people who are not very technical can use it easily. Apart from these main users, Researchers and policymakers can also benefit from the system. They can use the data on sales and demand to study buying habits, promote organic farming, and plan better policies. This opens the door for future improvements like smart farming tools or data-driven insights.

4.2 SOFTWARE ARCHITECTURAL DESIGN

4.2.1 ARCHITECTURAL DESIGN



4.1 Architectural Design for Organic farming System

The Organic Farming System follows the MVC design pattern, which divides the application into three parts: Models, Views, and Templates. Models manage all the important data such as farmer profiles, products, customer accounts, orders, and payments. They make sure the database always has correct and updated information. Views handle the logic of the system by connecting user actions with the data. For example, when a customer searches for a product, the view fetches the details from the model and shows it. Similarly, when a farmer adds or updates a product, the view processes the request and updates the system immediately.

Templates are the part that users see on their screen. They are designed to be simple and user-friendly, built with React.js to work smoothly on computers, tablets, and mobiles. Farmers get dashboards to manage their products, customers can browse and order items, and admins can track sales and reports. By separating the system into these three layers, the platform becomes

easier to maintain, secure, and reliable. It also makes it possible to add useful features like secure online payments, PDF receipts, analytics for admins, and notifications for users.

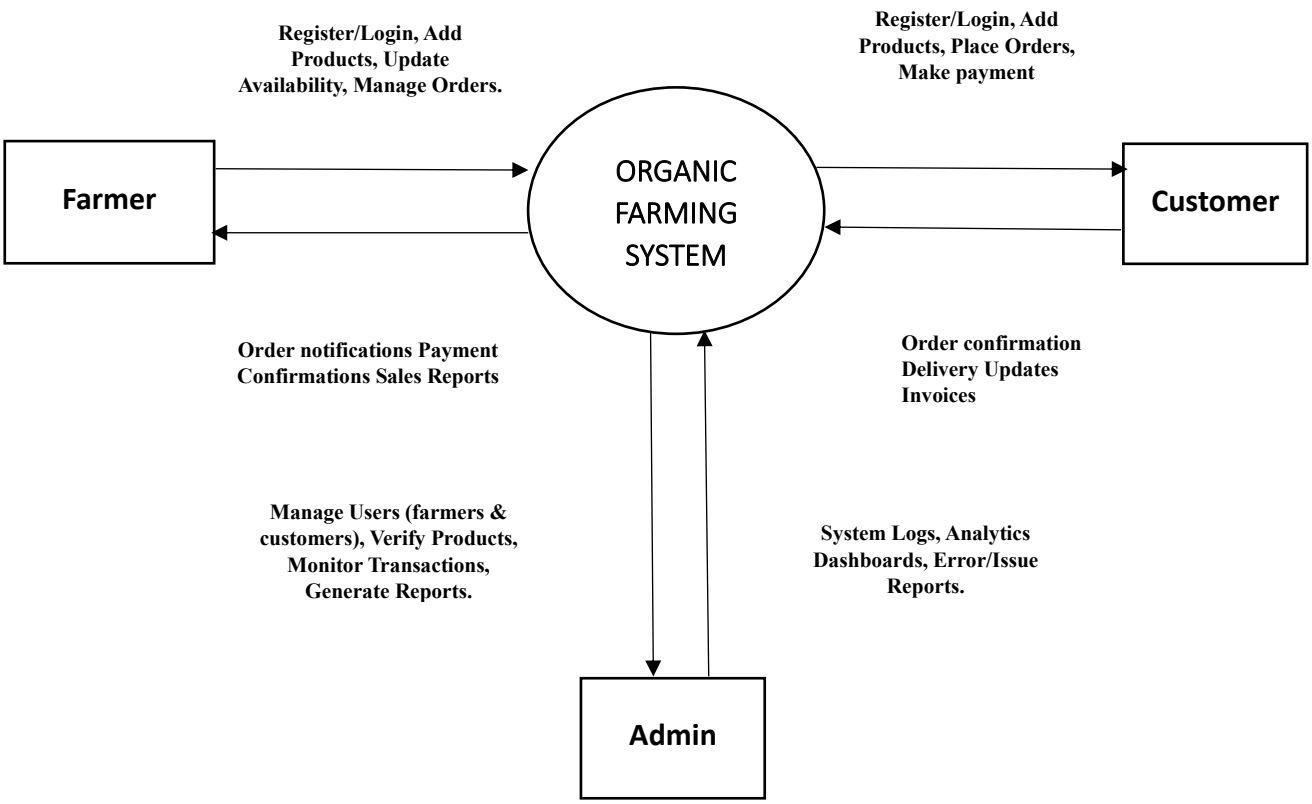
4.3 DATAFLOW DIAGRAM

An illustration of the flow of data through the Organic Farming System (OFS) is called a Data Flow Diagram (DFD). It gives developers and stakeholders a clear idea of how farmers, customers, and administrators interact with the system and how information is processed, stored, and shared among different groups. In OFS, DFDs illustrate inputs (like product uploads and order placements), processes (such as order management and payment handling), data stores (including users, products, orders, and payments), and outputs (like invoices, notifications, and reports). This provides a visual overview of how the platform operates.

Main components in the Data Flow Diagram:

- **Process**: Represents the core functions of the system, including Register/Login (Farmer, Customer, Admin), Add or Manage Products (Farmer), Browse and Place Orders (Customer), Process Payments, Generate Invoice, and Admin Monitoring. Each process checks inputs, applies business rules, and updates the relevant data stores.
- **Data Flow**: Shows how information moves between users, processes, and data repositories. For example, Product Details (Farmer → Add Product → Product Store), Order Request (Customer → Place Order → Order Store), Payment Confirmation (Customer → Payment Gateway → Payment Store), and Sales Report (Order Store → Admin Dashboard).
- **Data Store**: The system keeps collections like Users (farmer, customer, admin profiles with authentication), Products (organic crops, pricing, stock), Orders (customer purchase records, statuses), and Payments (transactions, invoices). These ensure data is secure, consistent, and quickly retrievable for real-time operations. \
- **External Entity:** Represents actors outside the system, such as Farmers (uploading products, updating deliveries), Customers (browsing products, placing orders, making payments), and Admins (monitoring analytics).

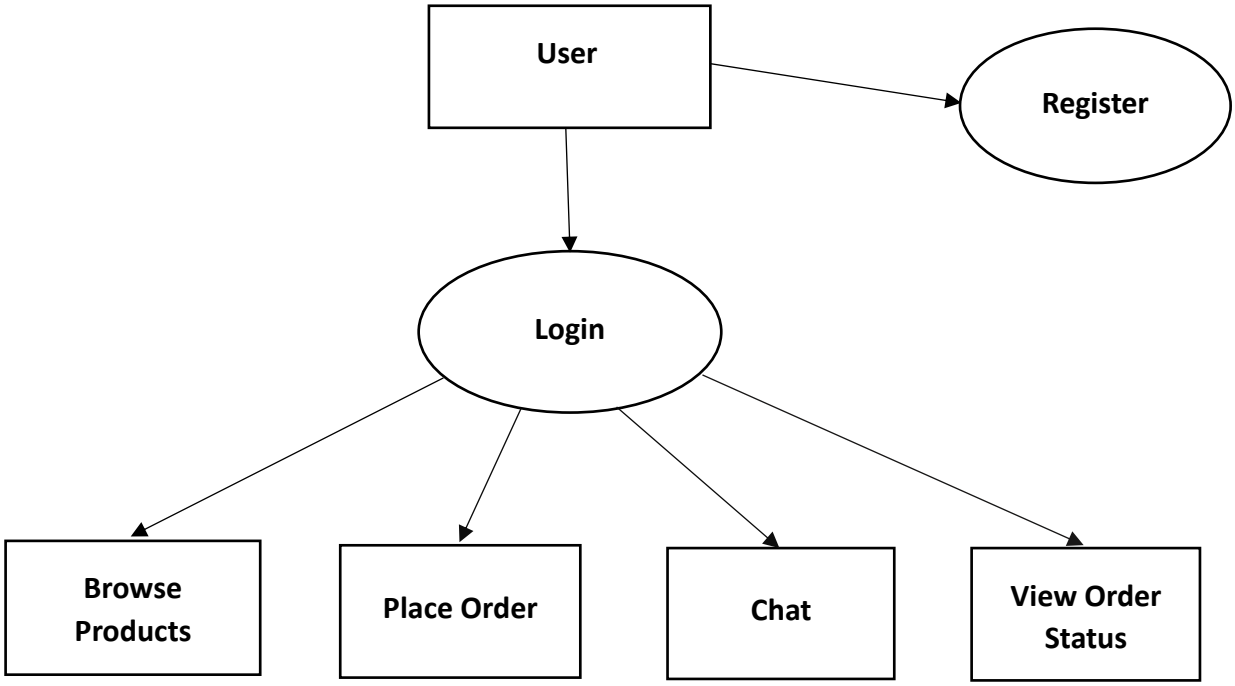
4.3.1 LEVEL-0 DFD FOR ORGANIC FARMING SYSTEM (CONTEXT FLOW DIAGRAM)



4.2 Context Diagram for Organic Farming System

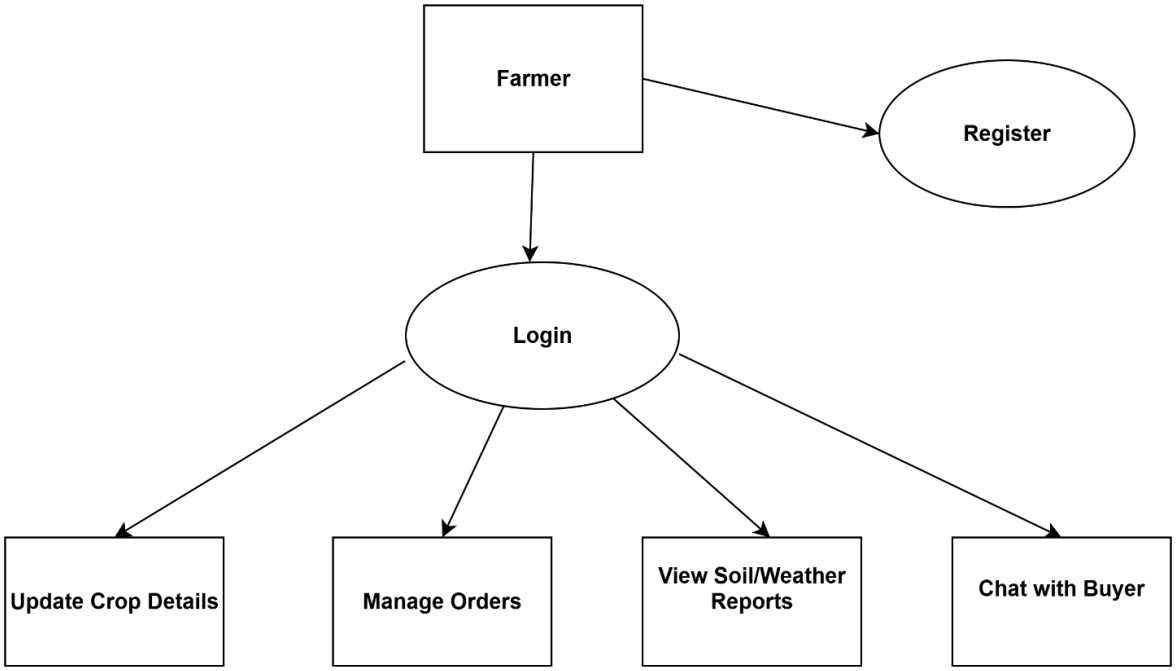
The context diagram of the Organic Farming System shows how farmers, customers, and admins interact with the platform. Farmers can register, add their organic products, update availability, and manage orders, while receiving notifications and payment confirmations. Customers can register, browse products, place orders, and make payments, and they receive order confirmations, delivery updates, and invoices. Admins oversee the system by verifying users and products, monitoring transactions, and generating reports. Overall, the system acts as a bridge that connects farmers and customers, with admins ensuring smooth and transparent operations.

4.3.1 LEVEL-1 DFD FOR USER



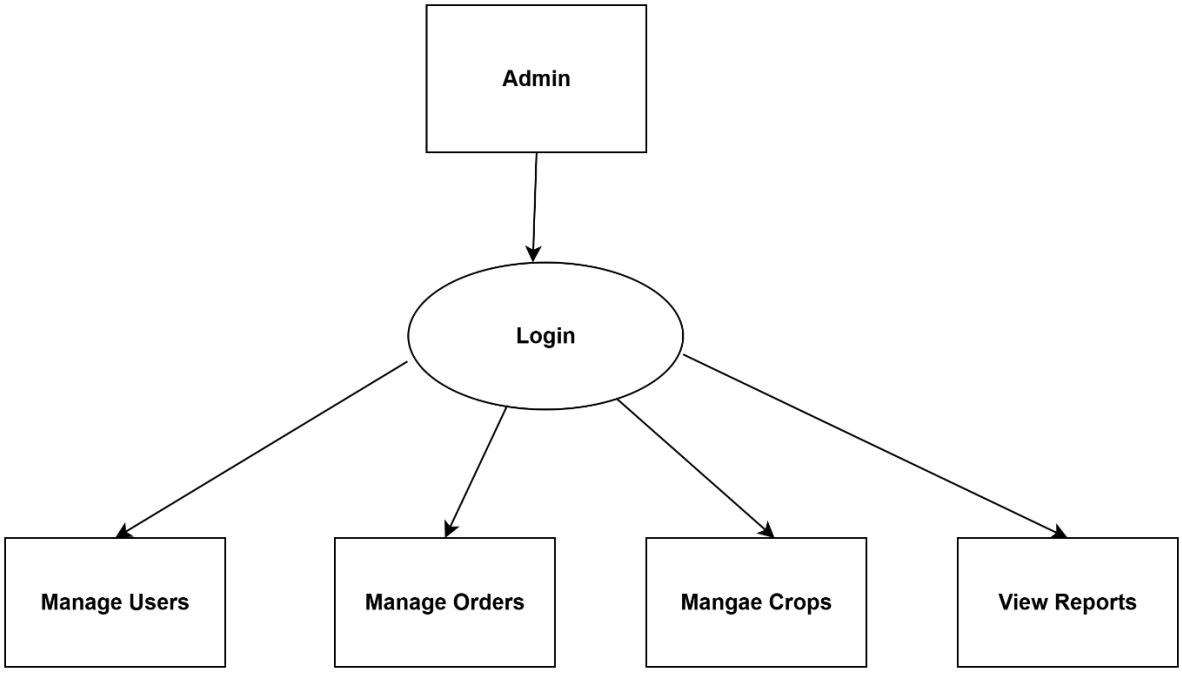
4.3 Level-1 DFD for USER

4.3.2 LEVEL-1 DFD FOR FARMER



4.4 Level-1 DFD for Farmer

4.3.2 LEVEL-1 DFD FOR ADMIN



4.5 Level-1 DFD for ADMIN

This diagram shows how the admin uses the system to keep everything organized. Just like other users, the admin first logs in to access the system. The administrator has complete control over all aspects of the platform once they are inside. They can oversee users by approving or keeping an eye on their actions, manage crop details to maintain the accuracy of the product information, and handle orders to ensure everything goes without a hitch. Additionally, the administrator has access to various reports that aid in their comprehension of the system's overall performance. In short, the admin acts like the supervisor who makes sure everything in the system works properly for both farmers and buyers.

CHAPTER 5

DETAILED DESIGN

5.1 INTRODUCTION

The Organic Farming System encourages the purchase and sale of organic products by bringing together farmers and consumers on a single platform. It makes it easier for farmers to display their produce, for consumers to peruse and purchase goods, and for system administrators to run the business effectively and openly. This chapter describes the system's architecture, data flow, and intermodular interactions in detail.

The design includes the following key areas:

- **Authentication**: To guarantee that only authorised users can access the system, farmers, customers, and administrators must securely log in and register.
- **Product Management**: Farmers have the ability to add, edit, or remove organic products, along with information about their name, category, cost, and description.
- **Farmer Profiles:** To gain customers' trust, farmers have profiles that detail their training, expertise, and certifications.
- **Shopping and Orders**: Clients have the ability to peruse merchandise, put items in their shopping cart, place orders, and monitor their progress.
- **Wishlist & Save for Later**: Clients have the option to save products they want to buy later.
- **Admin Management:** To guarantee the system's dependability, administrators validate farmers, authorise certifications, and keep an eye on transactions.

Along with presenting database schemas created using MongoDB, this chapter also discusses functional and non-functional requirements. When working with entities like users, products, orders, farmer profiles, and wishlists, MongoDB offers flexibility. The Organic Farming System's well-organised design guarantees that it functions efficiently and satisfies the demands of farmers, clients, and administrators in an approachable manner.

5.2 USE CASE DIAGRAM

A use case diagram, which is a UML diagram, illustrates the functions that various stakeholders can perform and how they interact with the Employee Attrition Analysis System. It describes

the system from a user perspective, emphasizing what the system does instead of how it works. This diagram helps capture system requirements, define system boundaries, and ensure all functions related to employee attrition prediction and analysis are well-organized and easy to understand for both technical and non-technical stakeholders.

Key Elements of the Use Case Diagram

The actors in the Organic Farming System represent external users interacting with the system. These include Farmers (who manage crops, update product details, and view order status), Customers (who place orders, peruse merchandise, make payments, and view order history), Administrators (who manage user accounts, oversee farmer and customer activities, and generate reports), and Delivery Personnel (who update delivery status and notify customers).

The use cases describe the fundamental features of the system, like adding and updating crop information, managing farmer and customer profiles, browsing and searching for products, placing orders, processing payments, tracking deliveries, generating invoices and reports, managing user roles, and recommending seasonal crops.

The system boundary, shown as a rectangle, includes all these functions under the Organic Farming System while keeping external actors outside. The relationships consist of connections between actors and use cases, include relationships for shared functions (for example, payment processing includes updating the order status), extend relationships for optional behaviors (for example, recommending seasonal crops), and generalization where specialized actors inherit common functions.

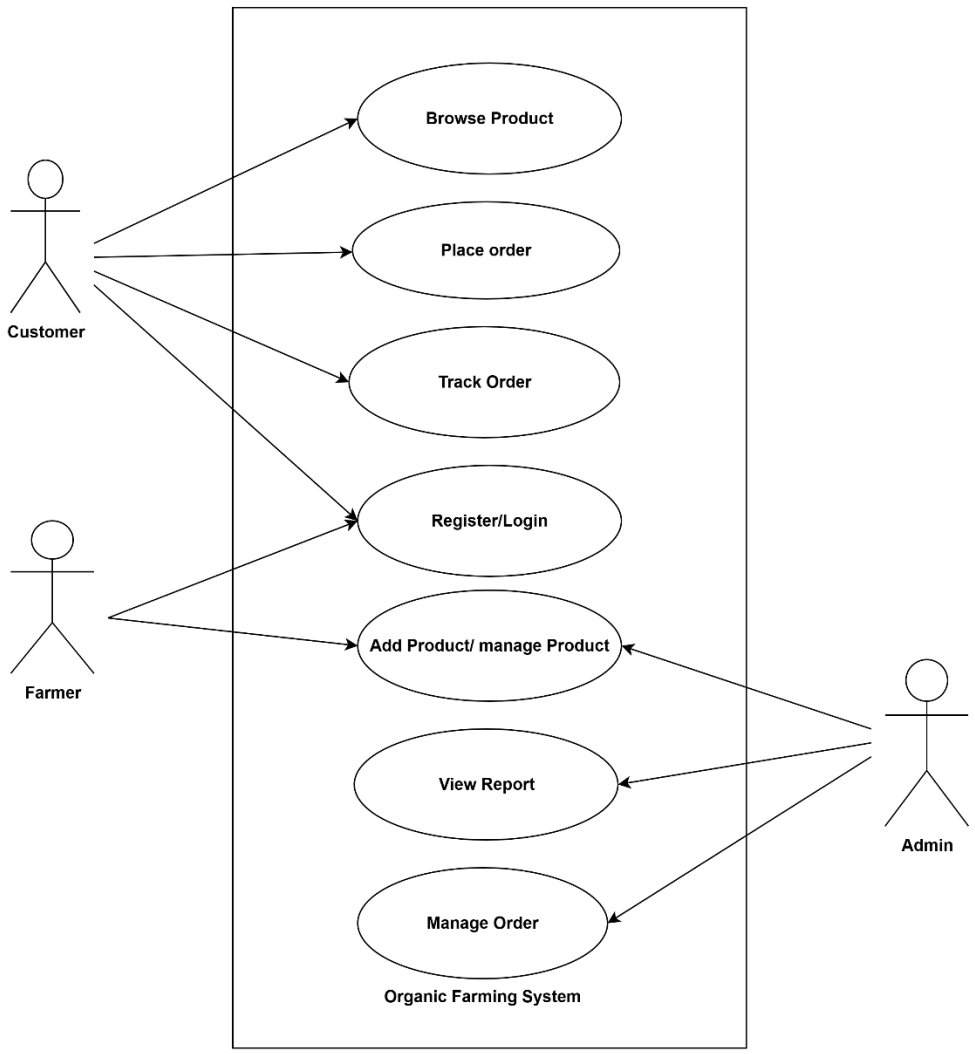


Fig 5.1 USE CASE DIAGRAM FOR ADMIN, CUSTOMER AND FARMER

5.3 SECQUENCE DIAGRAM

The order of actions and message exchanges between various actors and components in the Organic Farming System over time is depicted by a sequence diagram, which is a UML (Unified Modelling Language) interaction diagram. It visually represents how the system behaves and helps us understand how farmers, customers, and the system's internal modules work together to manage tasks like handling crops, placing orders, processing payments, and tracking deliveries.

- **Objects**: Objects represent the entities or modules involved in interactions. In the Organic Farming System, these objects include Farmer, Customer, Admin, Crop Management Module, Order Module, Payment Module, Delivery Module, Database, and External

Services (Payment Gateway/Email APIs). Each object has a vertical lifeline with its label at the top.

- **Activation Bar**: An activation bar is a thin vertical rectangle on an object's lifeline that shows how long it is active while performing a task. For example, when a Customer places an order, the Order Module becomes active until the request is validated, stored in the Database, and a confirmation is sent to the Customer.
- **Messages**: Messages are arrows between lifelines that represent communication or interaction. Synchronous messages (solid arrows) require an immediate response, such as when the Admin approves a crop listing or updates prices. Asynchronous messages (dashed arrows) do not need an immediate response, like when the system sends order confirmation emails or payment receipts.
- **Return Messages:** A return message is a dashed arrow pointing back to the sender, showing that a process has completed. For example, after a payment is processed through the Payment Gateway, the payment status returns to the Payment Module, which then updates the Database for the Customer and Admin.
- **Self-Messages:** A self-message occurs when an object sends a message to itself, represented with a loop arrow. For instance, the Crop Management Module may update stock quantities after a successful order before notifying the Customer.
- **Activation and Deactivation:** When an object starts a process, its activation bar begins. Once the process is complete, the activation ends. For example, when the Admin approves a farmer profile, the Farmer Management Module becomes active during verification. Once the changes are stored in the Database, it deactivates.

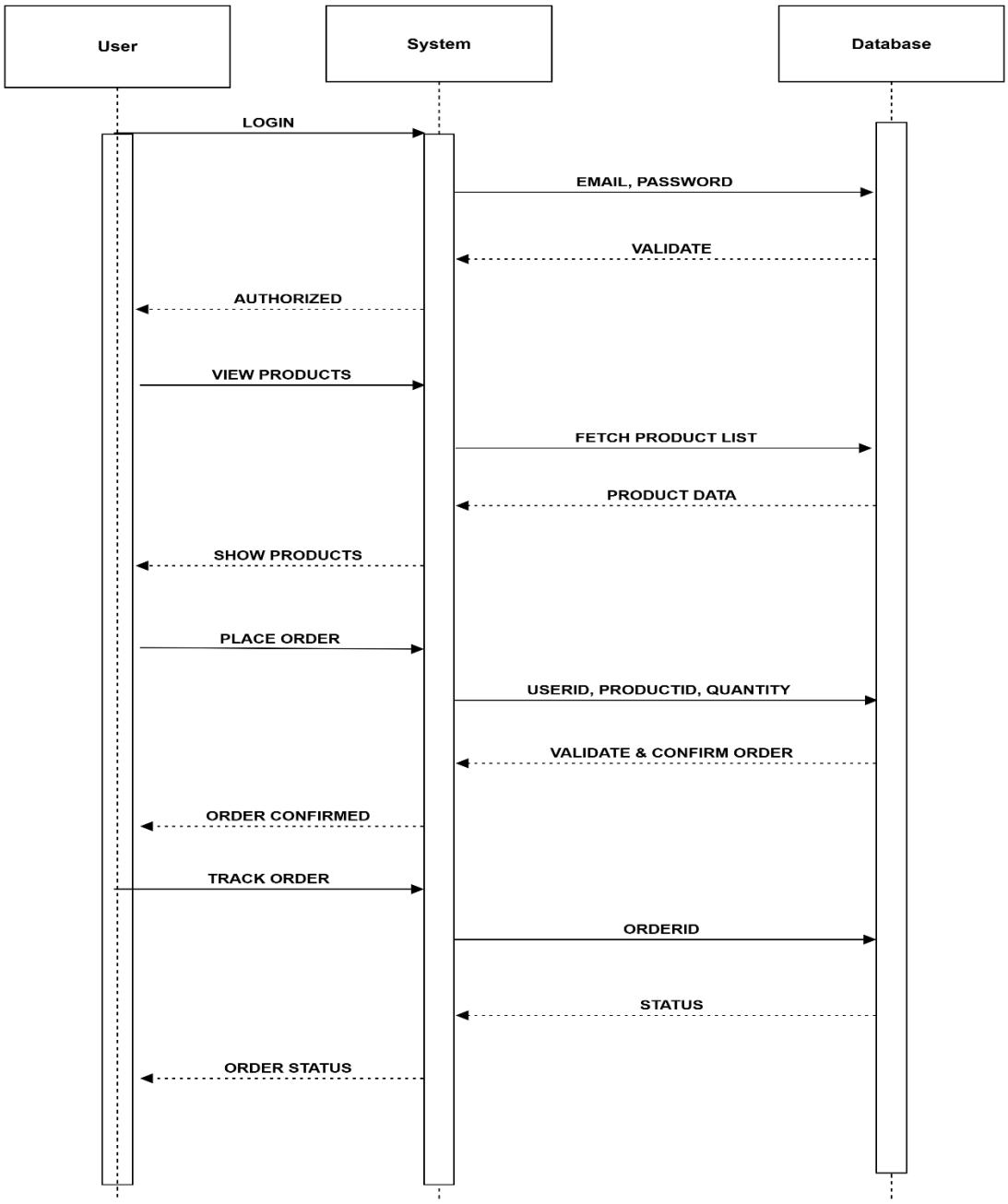


Fig. 5.2: SECQUENCE DIAGRAM FOR USER

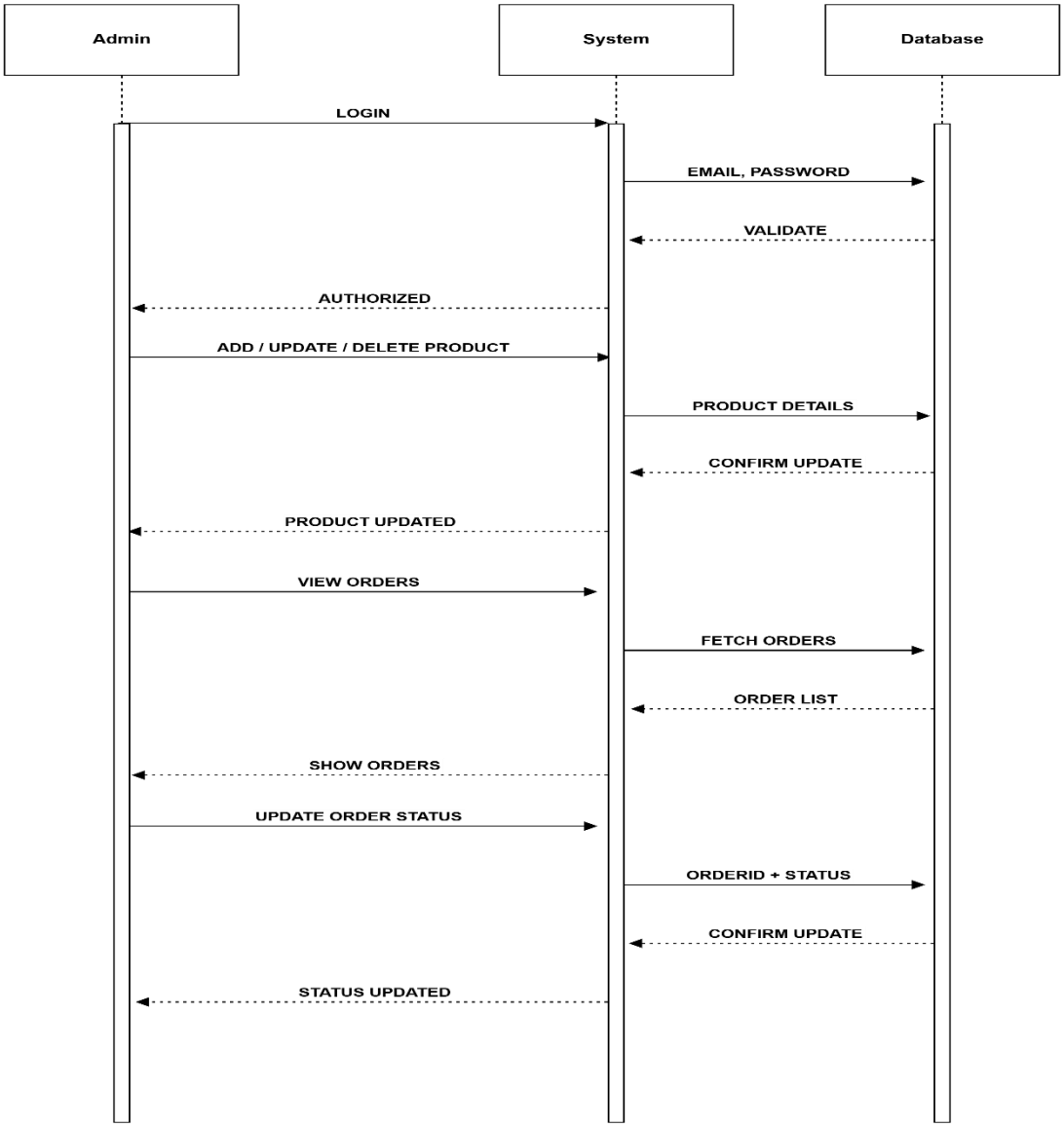


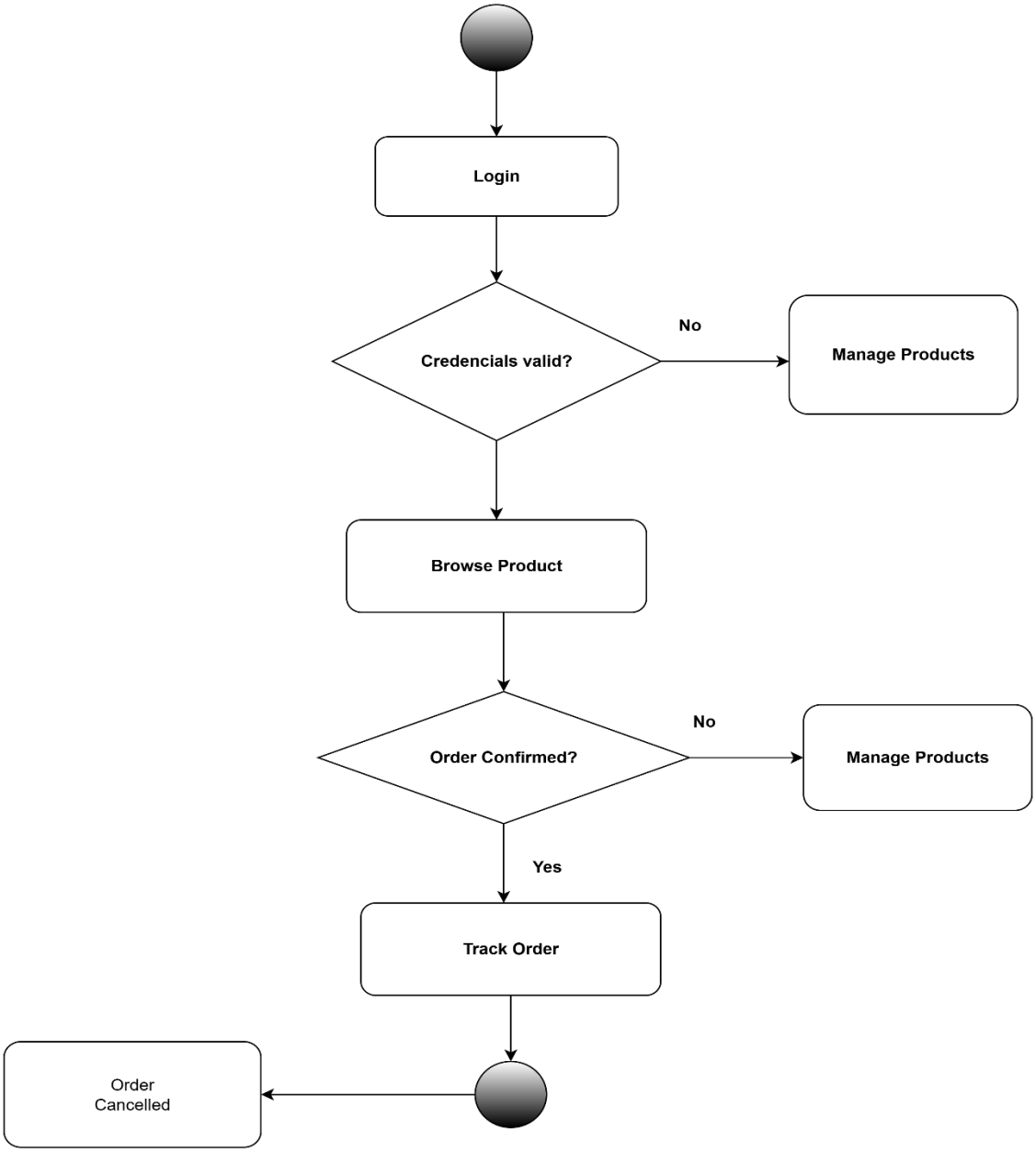
Fig. 5.3: SECQUENCE DIAGRAM FOR ADMIN

5.4 ACTIVITY DIAGRAM

In the Organic Farming System, the activity diagram illustrates how the system operates step by step. It shows how different users, such as customers, farmers, and admins, interact with it. For customers, the flow typically begins with logging in. Then, they can browse available products, place an order, and later track the status of their order. Along the way, there are decision points, such as whether the login details are correct or if the order is confirmed.

For admins or farmers, the diagram outlines tasks like adding new products, updating stock, or managing customer orders. Customers will always see the most recent information about

what is available thanks to this. Additionally, the diagram shows that certain activities take place concurrently, like multiple customers placing different orders while the admin updates the product list. By examining the activity diagram, everyone can easily grasp the complete flow of the system—from login to order delivery—ensuring the process is clear, smooth, and well-organized.



5.4 Activity Diagram for User

5.5 DATABASE DESIGN

The Organic farming System contains numerous tables, but the one presented here are the main crucial and relevant to the system.

Table 5.1 Admin

Sl. No	Field Name	Data Type	Size	Constraints	Description
1	username	varchar	100	unique, not null	Admin's login username
2	password	varchar	255	not null	Admin's login password (hashed before storing)

Table 5.2 User

Sl. No	Field Name	Data Type	Size	Constraints	Description
1	UserId	varchar	24	Primary key	A distinct identity for every user
2	username	String	100	Unique, Required	User's name
3	email	String	100	Unique, Required	User's email ID
4	password	String	20	Required	User password (hashed)
5	otpHash	String	20	Optional	OTP for email verification
6	isEmailVerified	Boolean	—	default = false	Indicates if email has been verified

Table 5.3 Product

Sl No	Field Name	Data Type	Size	Constraints	Description
1	ProductId	ObjectId	100	Primary key	Unique product ID
2	name	String	100	Required	Name of the product
3	price	int (FK)	11	Required	Price of the product
4	category	String	500	Optional	Product category (e.g., Vegetables, Fruits)
5	description	String	100	Optional	Details about the product
6	imageUrl	String	100	Optional	URL of the product image

Table 5.4 Order

Sl No	Field Name	Data Type	Size	Constraints	Description
1	orderId	objectId	11	Primary key	Unique order ID
2	userEmail	String	—	Required	Email of the customer placing the order
3	products	varchar	100	Required	List of ordered products
4	total	int	100	Required	Total price of the order
5	customerName	String	—	Required	Full name of the customer
6	address	String	—	Required	Delivery address
7	paymentMethod	String	—	Required	Payment option selected
8	tracking	Array of SubDocs	—	Optional	Status history

Table 5.5 FarmerProfile

Sl No	Field Name	Data Type	Size	Constraints	Description
1	farmerId	int	11	Primary key	Unique farmer profileID
2	displayname	String	—	Required	Farmer's display name
3	bio	String	—	optional	Short description
4	location	String	—	optional	Farmer's location
5	certificates	Array<SubDoc>	100	Optional	Certification documnents uploaded by farmer
6	photoUrl	String	—	Optional	Path/URL to farmer's profile photo

5.6 ER DIAGRAM

An Entity-Relationship (ER) diagram for the Organic Farming System shows the logical structure of the database. It demonstrates how different entities, such as users, farmers, products, orders, and admins, are connected. This diagram acts as a design guide, ensuring smooth database implementation and clear understanding of relationships among data

components. The ER diagram is a key part of database design because it helps cut down on redundancy, keep data consistent, and make interactions between system components simpler. It also serves as a communication tool between database designers, developers, and stakeholders, ensuring everyone understands before the actual implementation.

The main components of an ER diagram are:

- **Entities**: Entities are the main objects in the system that hold data about real-world concepts. In the Organic Farming System, the main entities are User, Farmer, Product, Order, and Admin. Every entity has distinct qualities that characterise it. For example, the User entity holds details like name, email, and phone number, while the Product entity records information such as product name, price, category, and description. These entities are the foundation of the database, representing the key participants in the system.
- **Attributes**: Attributes describe the properties or characteristics of each entityThey aid in defining the specifics and uniqueness of the data that is stored. A user, for example, has attributes such as name, email, phone number, and userId. A Farmer entity includes attributes such as farmerId, displayName, bio, and location, while a Product has productId, price, category, and availability. Some attributes are simple, like price, while others can be composite, such as an address that includes street, city, and pincode. Attributes ensure that entities provide enough detail to capture all necessary information.
- **Primary Key**: To differentiate one record from another, each entity is given a unique identifier known as the primary key. Each entity in the Organic Farming System has its own primary key, such as orderId for Order, farmerId for Farmer, productId for Product, and userId for User. These keys are vital for maintaining record uniqueness and ensuring accurate data retrieval without duplication. By enforcing uniqueness, the primary key keeps the database reliable and consistent.
- **Relationships**: Relationships represent the connections between entities and explain how they interact. In this project, several relationships exist: a User places an Order, a Farmer supplies Products, an Order contains Products, and an Admin verifies a Farmer. These relationships ensure that entities are connected in a meaningful way, reflecting real-world interactions in the organic farming system.
- **Cardinality**: Cardinality defines how many instances of one entity can be associated with instances of another entity. In the Organic Farming System, one User can place

many Orders (1:N), one Farmer can supply many Products (1:N), One Product may appear in multiple Orders (M:N), and multiple Products may be included in a single Order. Cardinality aids in accurately modelling the system's business rules.

- **Foreign Key**: By making reference to another entity's primary key, foreign keys establish a connection between entities. For instance, the Product entity has farmerId referring to the Farmer entity, and the Order entity has userEmail or userId as a foreign key referring to the User entity. By maintaining related data consistent across entities, these connections guarantee referential integrity.
- **Participation**: Participation specifies whether all or only a relationship involves some instances of an entity. In the Organic Farming System, every Order must be linked to a User (total participation), while a Product may or possibly not linked to a Farmer (partial participation, if the admin adds it directly). Similarly, every Farmer needs to be confirmed by an Admin before they can sell products, which shows total participation.

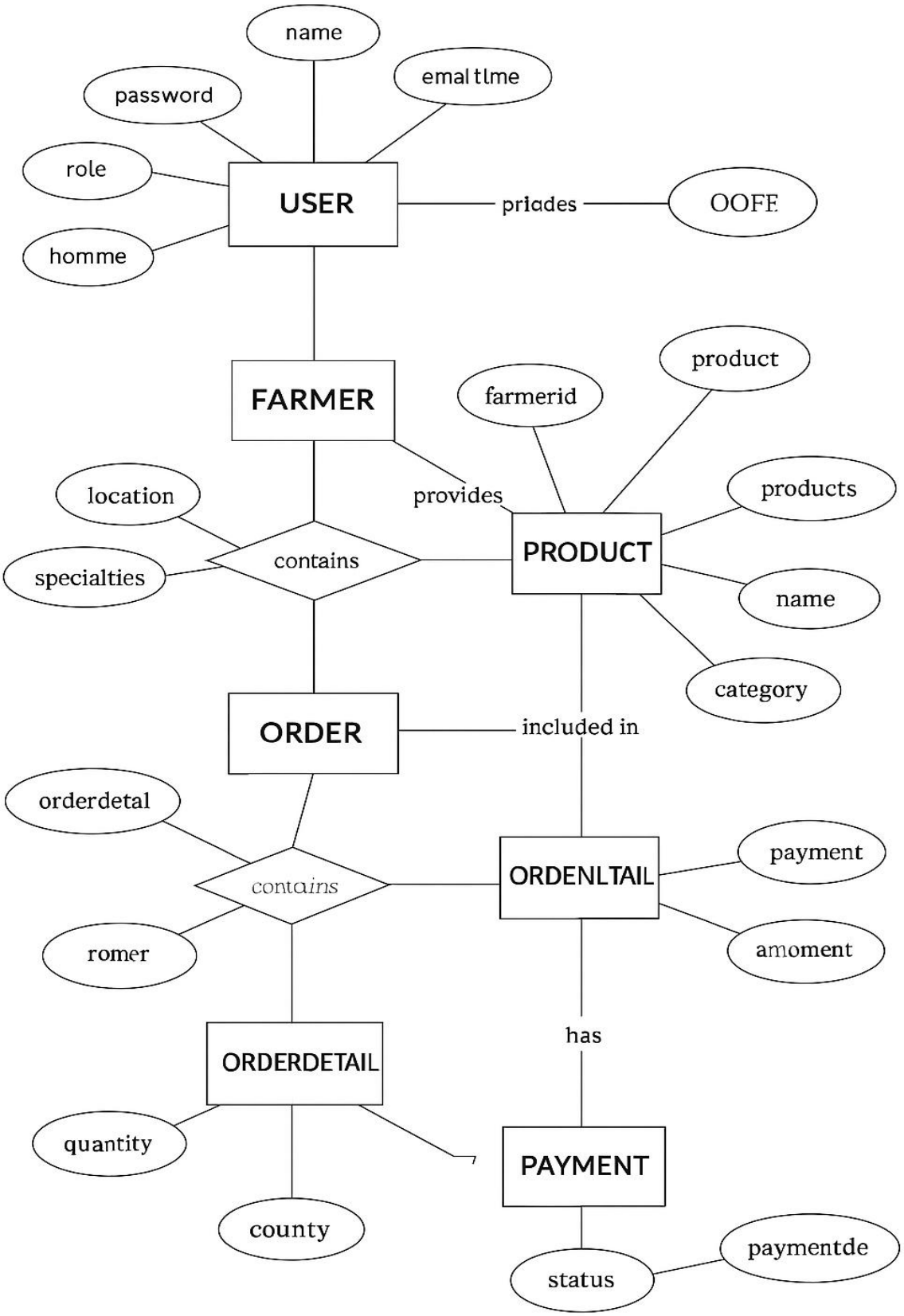


Fig. 5.5 ENTITY RELATIONSHIP DIAGRAM

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

The stage of implementation is when the intended system is constructed and made operational. Here, the database, modules, and designs are all integrated to produce a functional system. Making sure it functions correctly and benefits farmers and consumers is the goal.

In the Organic Farming System, the backend is built with Node.js and Express.js to handle things like login, orders, and product management. The frontend uses React.js to give farmers and customers a simple and easy interface. Data such as profiles, products, and orders is stored in MongoDB. Additionally, the system has functions like farmer verification, product listings, secure payments, and order tracking. With real-time updates through WebSockets, users get instant information. Together, these technologies make the system easy, reliable, and helpful for connecting farmers and customers.

6.2 USER INTERFACE DESIGN

The User Interface (UI) is the part of the system that farmers, customers, and admins see and use directly. A simple and clear design is important because it makes the system easy to understand and helps users do their tasks without confusion. Farmers can quickly list or update their products, while customers can browse, order, and track their organic items smoothly.

In the Organic Farming System, the UI is created using React.js for a clean and responsive design, while HTML5 and CSS are used for layout and styling. This makes the platform work well on computers, tablets, and mobile phones. Even those with limited technical expertise can easily use it thanks to the design's emphasis on simplicity and usability. The interface includes farmer profiles, product listings with images, a shopping cart, and order tracking. Real-time updates and notifications keep users informed, and feedback helps improve the system over time.

.

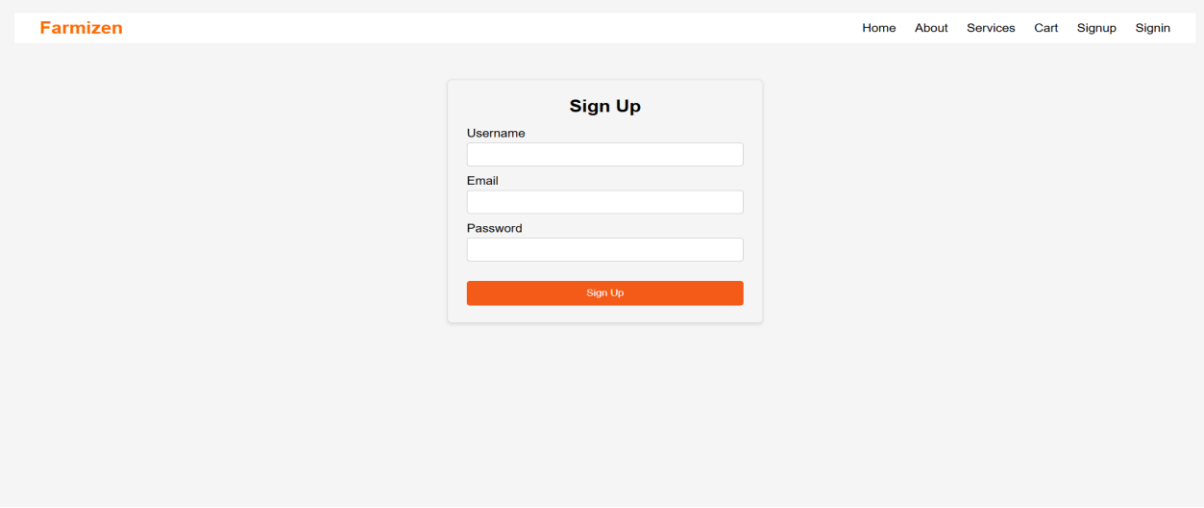


Fig. 6.1 SignUp Page

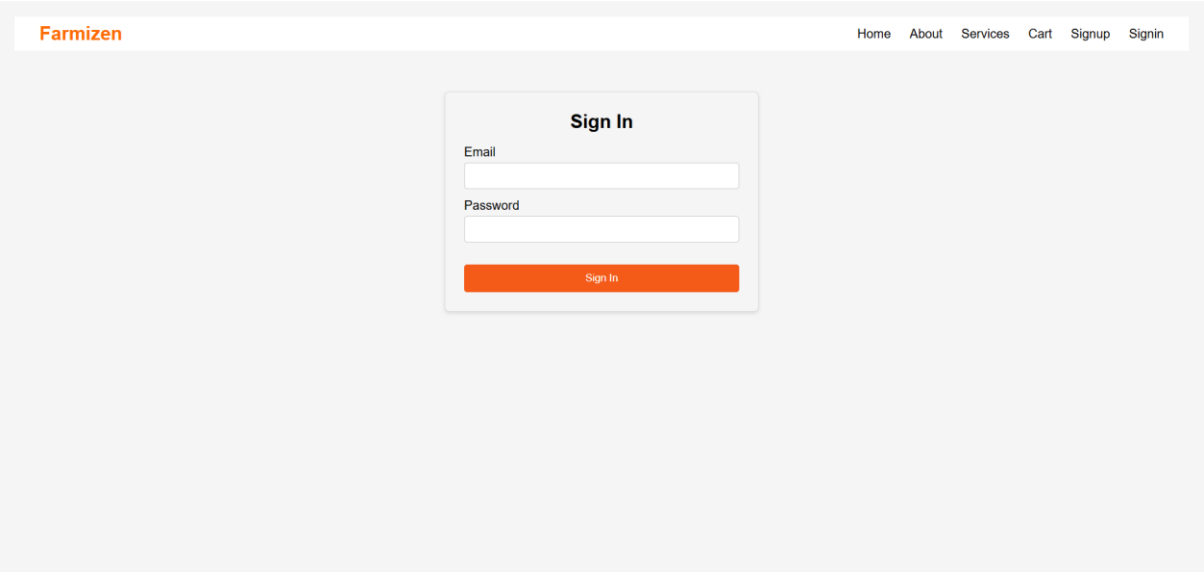


Fig. 6.2 Sign In Page

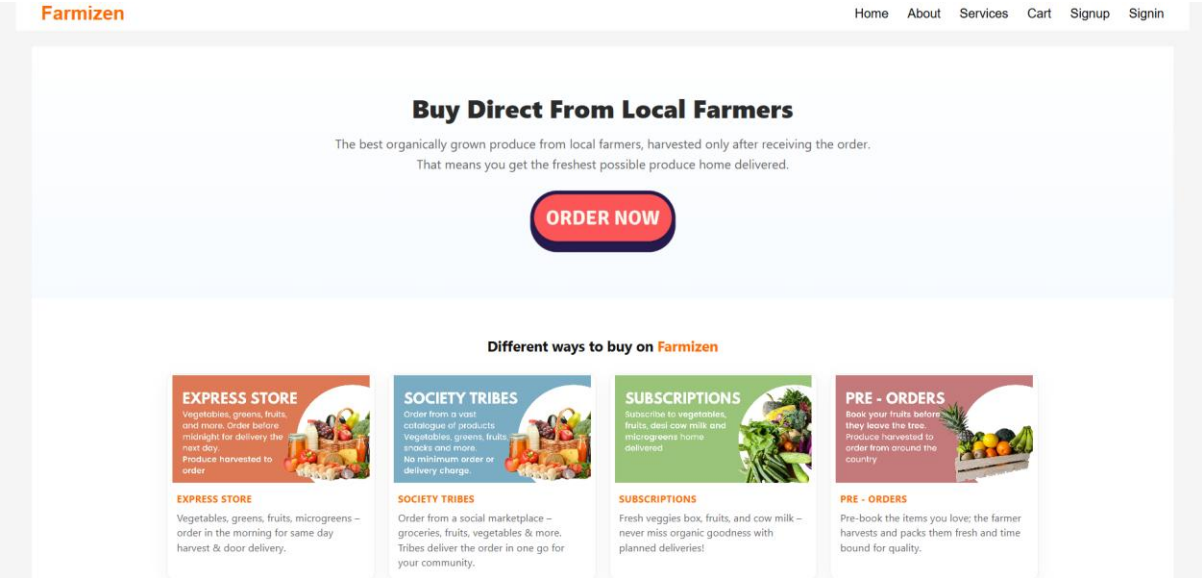


Fig. 6.3 Home page

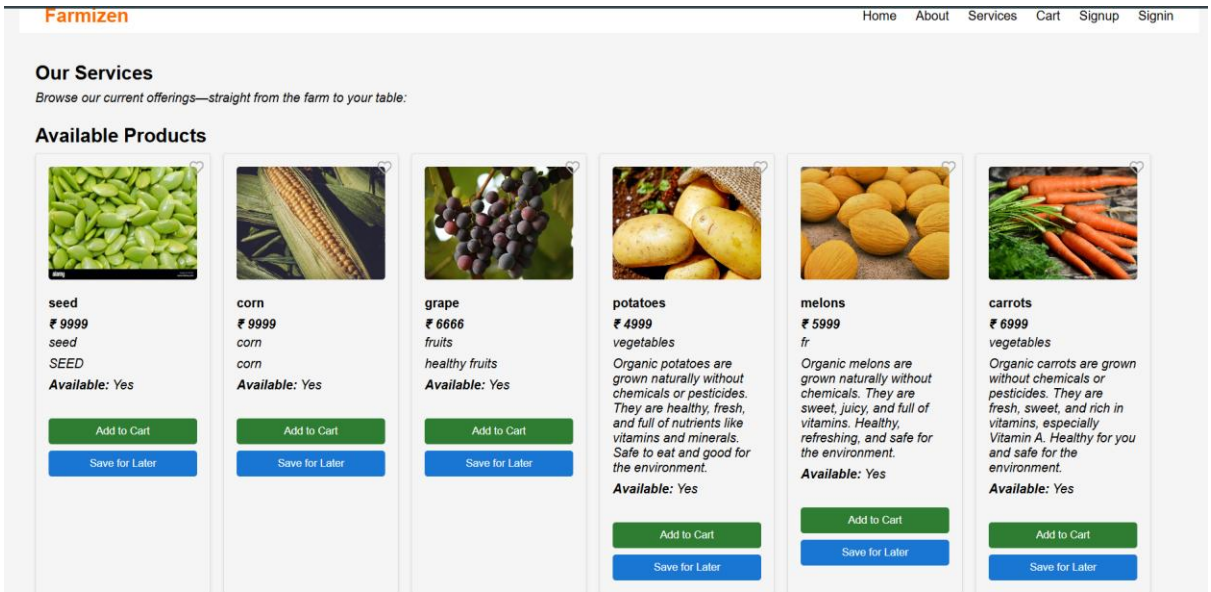


Fig. 6.4 Service Page

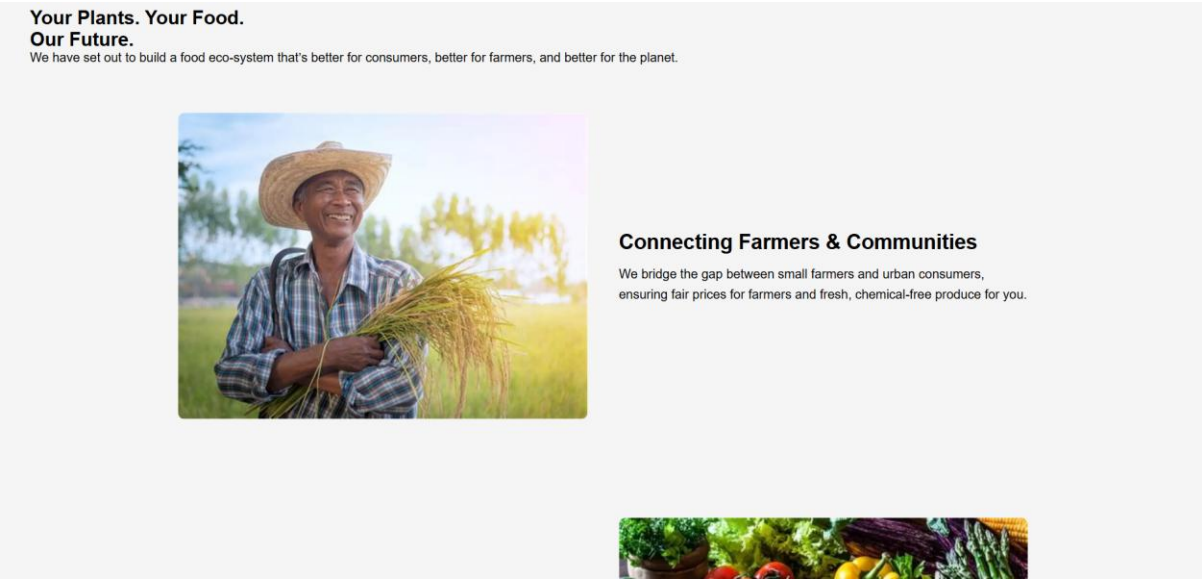


Fig. 6.5 About Page

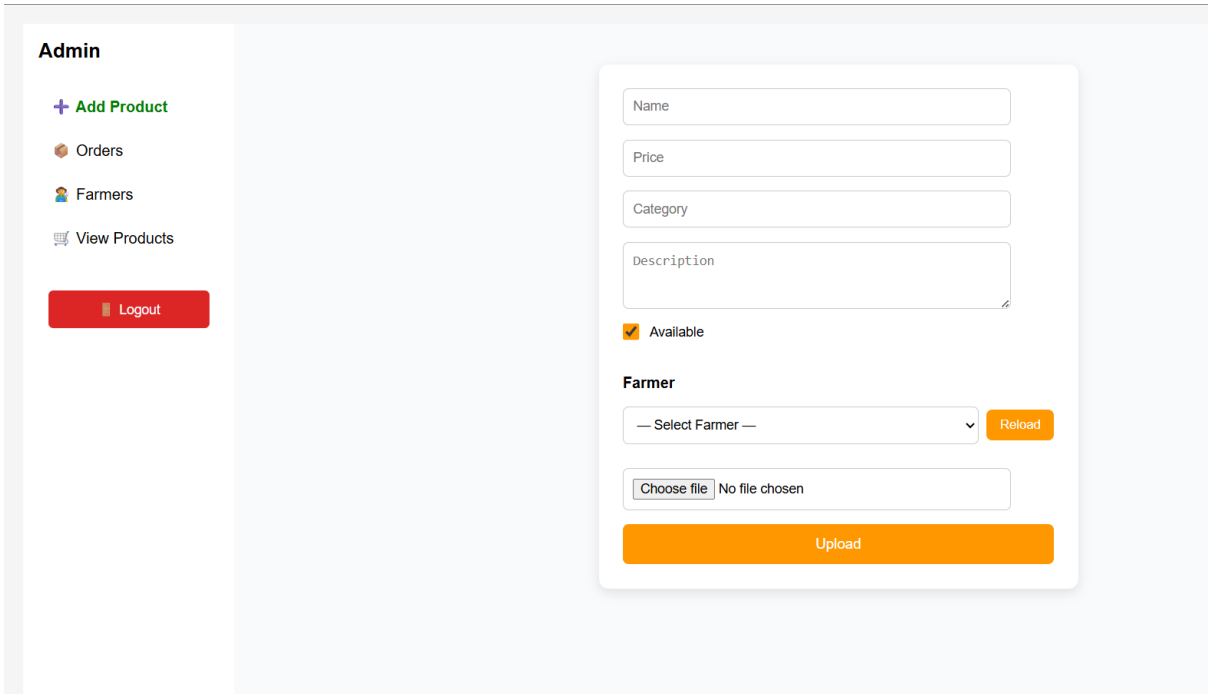


Fig. 6.6 Admin Page

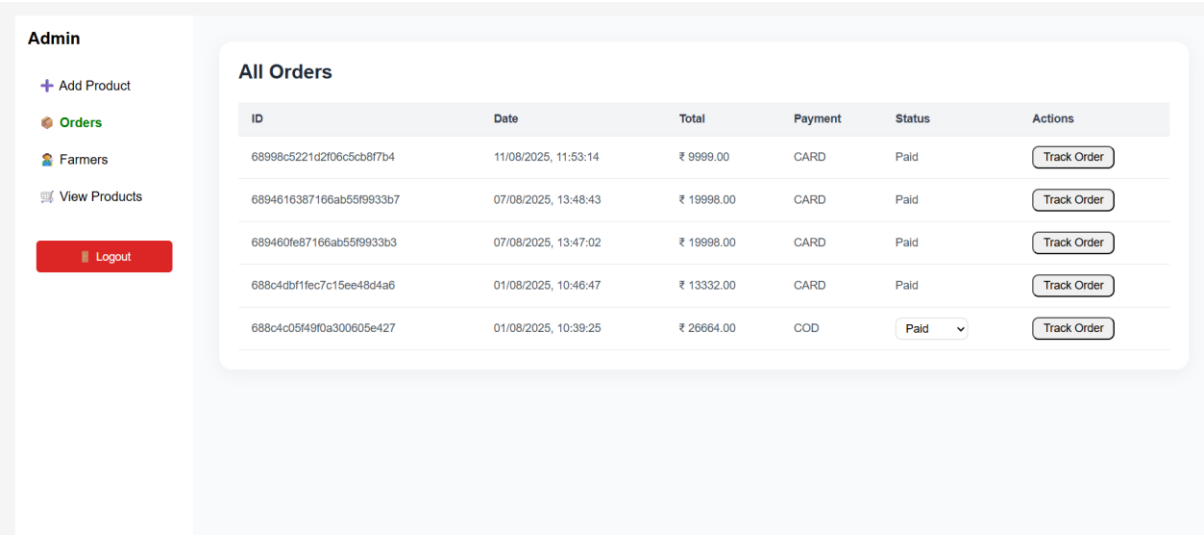


Fig. 6.7 Admin Tracking Pages

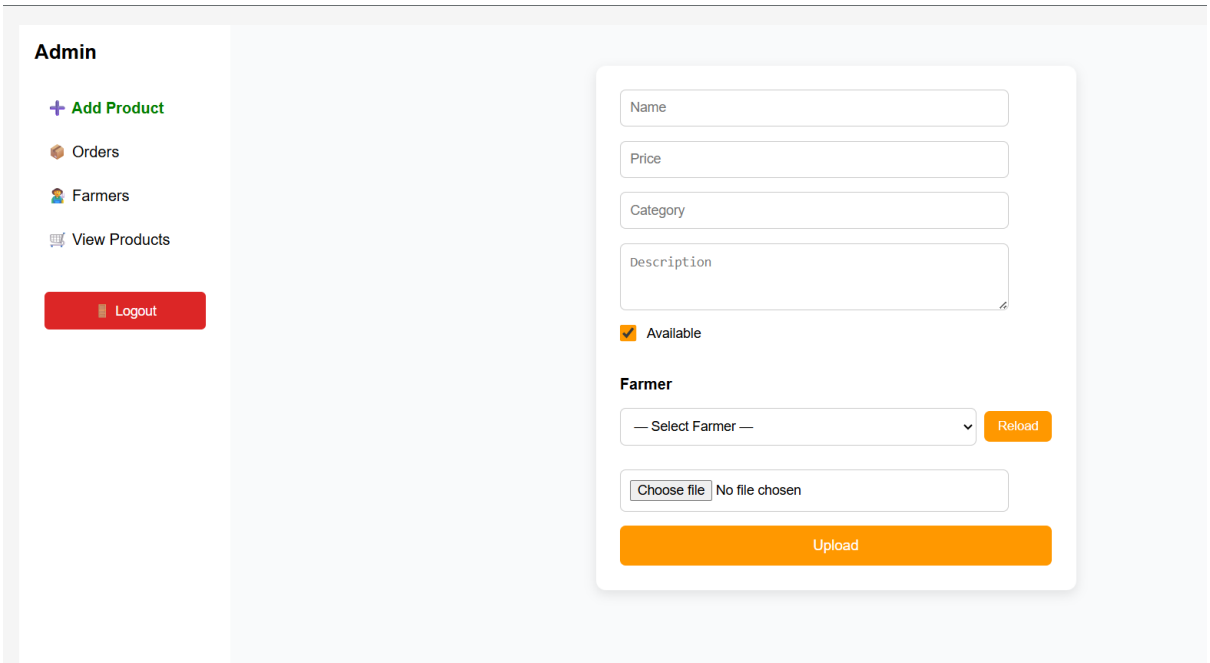


Fig. 6.8 Admin Add Product Pages

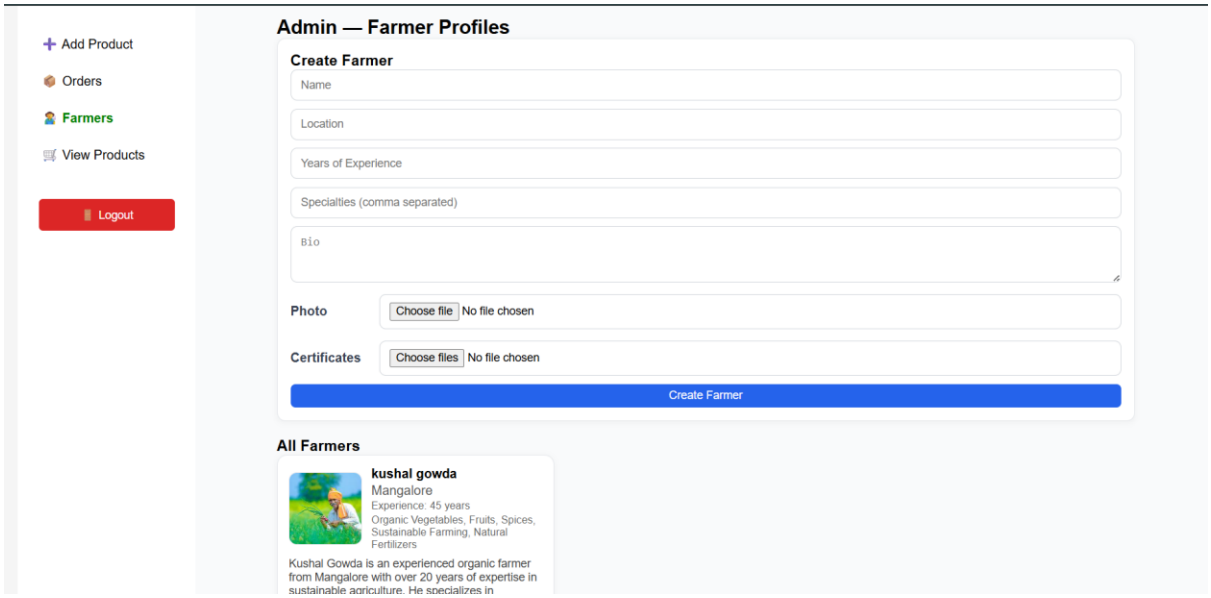


Fig. 6.9 Admin- Farmer profile Page

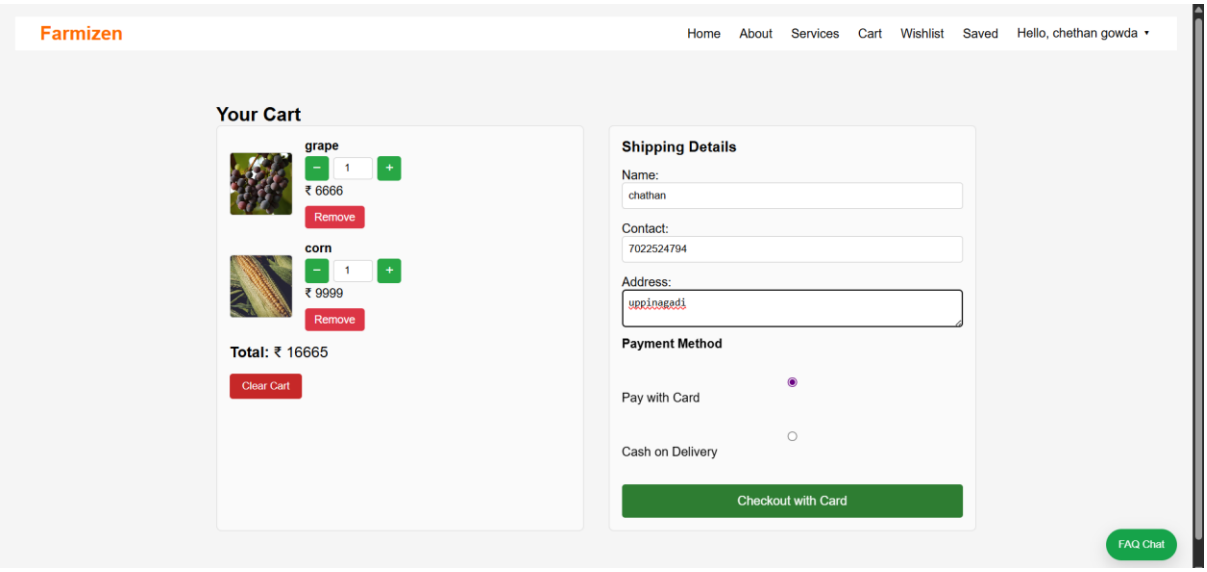


Fig. 6.10 Order pages

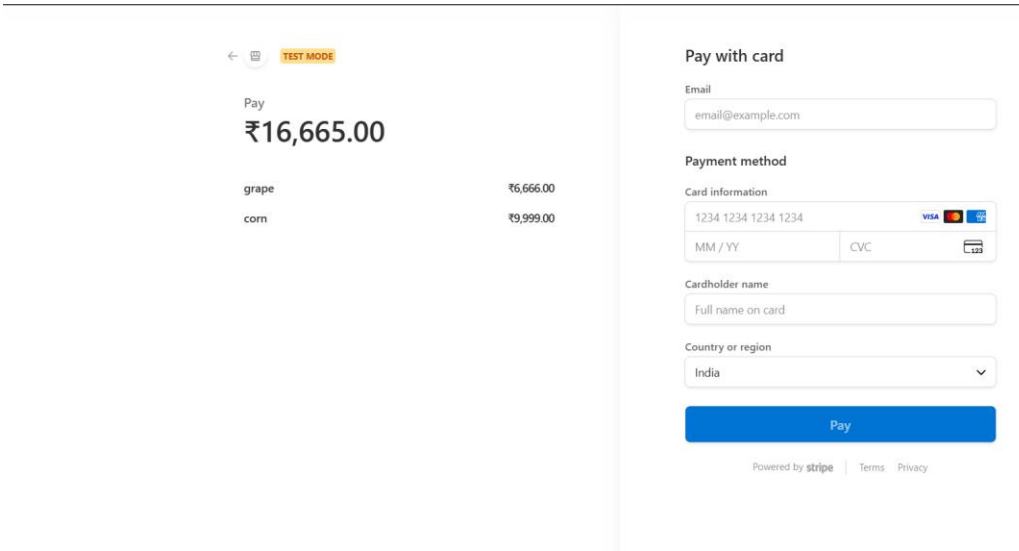


Fig. 6.11 Payment Page

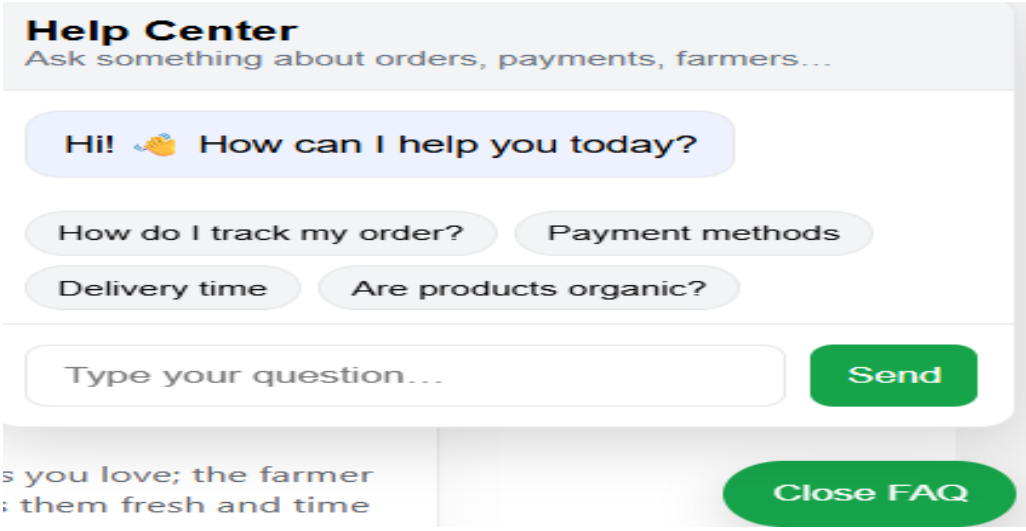


Fig.6.12 Chat page

CHAPTER 7

SOFTWARE TESTING

7.1 INTRODUCTION

Testing is an important step in building the Organic Farming System because it checks if everything works the way it should. Each feature, like user login, product listing, order handling, payments, and weather updates, is tested carefully. This makes sure the system runs smoothly, gives correct results, and avoids errors. We also test how the system behaves when many farmers and customers use it at the same time. This is called performance testing, and it helps the system stay fast and reliable.

Another key part is security testing, which protects user data, payments, and other private information from misuse. Finally, we do User Acceptance Testing (UAT), where real farmers and customers use the system and share feedback. This helps improve the system and make it more user-friendly. By testing functionality, performance, security, and user feedback, we ensure the Organic Farming System is safe, reliable, and ready for real-world use.

7.2 TESTING CRITERIA

To ensure that the Organic Farming System (OFS) functions well and meets the needs of farmers, buyers, and administrators, we set clear testing criteria. These criteria help us assess how well the system operates, its reliability, and its usability.

- **Functional Accuracy:**

  The OFS must execute all its primary tasks accurately. This involves user registration, login, accessing crop suggestions, checking weather updates, listing or browsing products in the marketplace, placing orders, and participating in forums. Each feature is tested in various situations to ensure that it works effectively and produces correct results.

- **Integration Cohesion:**

  All components of the OFS should function together without issues. Data should flow correctly between the database, user interface, and external services, like weather APIs. For

example, a crop suggestion based on soil data should be shown accurately to the user, and product listings must update correctly across buyer and farmer views.

- **Performance Efficiency:**

  The system should perform well, even when many users access it simultaneously—for instance, when several farmers are placing orders or checking market prices at once. We conduct tests to evaluate the speed and stability of the system under pressure, along with its resource usage.

- **Security Robustness:**

  Security is essential, particularly when handling payment information, farming data, and personal information. We test to ensure that data is secured from unauthorized access and that the system is protected from common threats or attacks.

- **Usability and User Experience:**

  Whether users are farmers, buyers, or administrators, the platform should be simple, easy to use, and intuitive. Testing verifies that the design is user-friendly across different devices, and we gather feedback to facilitate improvements based on real user interactions with the system.

- **Reliability and Stability:**

  The OFS should operate without crashing or showing unexpected behavior. If problems arise, the system should manage them smoothly without losing data. We carry out long-term testing to ensure the system remains stable and reliable over time.

- **Compliance and Standards:**

  The system must abide by all applicable laws and standards, including those pertaining to online payments, digital accessibility, and data protection. We conduct tests to confirm that the OFS meets these legal and technical requirements, providing users with a safe and trustworthy experience.

7.3 TESTING METHODOLOGIES

1. Functionality Testing

Functionality testing ensures that every feature of the Organic Farming System works as planned. This includes modules like farmer registration, crop management, organic certification tracking, marketplace transactions, inventory management, weather alerts, and report generation. We simulate real-world farming and trading situations in test scenarios to confirm that the system produces accurate and reliable outputs under expected conditions.

2. Integration Testing

Evaluations for testing integration how different components of the OFS function together as a unified platform. It ensures smooth data exchange between farmers, buyers, certification authorities, crop records, transactions, and the database. By identifying inconsistencies in module interactions early, integration testing helps make certain that the system operates seamlessly as a whole.

3. User Interface (UI) Testing

UI testing focuses on verifying that the OFS is user-friendly, responsive, and available on a range of devices, such as smartphones and tablets used by farmers. We assess the interface for clarity, flow of navigation, and multilingual support to guarantee that farmers, buyers, and administrators can use the platform effectively without confusion.

4. Performance Testing

Testing for performance analyses how the OFS responds under different workloads, such as during peak harvesting seasons when numerous farmers upload data, buyers place orders, and transactions occur simultaneously. We test key aspects like system response time, scalability, and resource use. This guarantees that during crucial farming times, the system will continue to be dependable, quick, and able to manage large transaction volumes.

5. Security Testing

Security testing protects sensitive agricultural and financial data within the OFS. We look for threats like misuse of data, online payment vulnerabilities, and illegal access. Our main goal was safeguarding farmer records, marketplace transactions, organic certification details, and financial information, thereby ensuring trust, reliabilityas well as adherence to data protection guidelines.

7.4 TEST CASES

Table 7.1 – Test Cases for Farmer Registration

No	Test Case	Expected Output	Observed Output	Result
1	Name field left empty	Alert: "Please fill all fields before submitting"	Alert displayed	Pass
2	Email field left empty	Alert: "Please fill all fields before submitting"	Alert displayed	Pass
3	Invalid email format entered	Alert: "Enter a valid email address"	Alert displayed	Pass
4	Password and Confirm Password do not match	Alert: "Passwords do not match"	Alert displayed	Pass

Table 7.2 – Test Cases for Farmer Login

No	Test Case	Expected Output	Observed Output	Result
1	Login with empty Email & Password	Alert: "Please enter valid email and password"	Alert displayed	Pass
2	Incorrect email entered	Alert: "Invalid Email ID"	Alert displayed	Pass
3	Incorrect password entered	Alert: "Invalid Password"	Alert displayed	Pass
4	Correct login credentials entered	Redirect to Farmer Dashboard	Dashboard displayed	Pass
5	Server not responding	Alert: "Unable to connect. Try again later."	Alert displayed	Pass

Table 7.3 – Test Cases for Certification Management

No	Test Case	Expected Output	Observed Output	Result

1	Submit application with missing details	Alert: “Please fill all fields”	Alert displayed	Pass
2	Submit invalid certification details	Alert: “Invalid details entered”	Alert displayed	Pass
3	Submit valid certification request	Alert: “Application submitted successfully!”	Success message displayed	Pass
4	Admin approves certification	Certification marked as approved	Approved successfully	Pass
5	Server error while processing certification	Alert: “Error processing request”	Alert displayed	Pass

Table 7.4 – Test Cases for Weather & Alert Notifications

No	Test Case	Expected Output	Observed Output	Result
1	Open weather forecast screen	Weather details displayed correctly	Data displayed	Pass
2	Attempt to fetch weather data with no internet	Alert: “Unable to fetch weather updates”	Alert displayed	Pass
3	Receive crop disease alert	Notification displayed to farmer	Alert displayed	Pass
4	Refresh weather updates	Updated data displayed	Refreshed correctly	Pass
5	Server error while fetching alerts	Alert: “Error fetching alerts”	Alert displayed	Pass

Table 7.5 – Test Cases for Admin Dashboard

No	Test Case	Expected Output	Observed Output	Result

1	Admin login with valid credentials	Dashboard loads with stats	Dashboard displayed	Pass
2	Admin login with invalid credentials	Alert: “Invalid login credentials”	Alert displayed	Pass
3	Add bus successfully	Bus added to database	Bus displayed in list	Pass
4	Assign route to bus	Route assigned successfully	Success message displayed	Pass
5	Server error while fetching schedules	Alert: “Error fetching data”	Alert displayed	Pass

Table 7.6 – Test Cases for Marketplace Transactions

No	Test Case	Expected Output	Observed Output	Result
1	Attempt to purchase with empty cart	Alert: “Your cart is empty”	Alert displayed	Pass
2	Invalid payment details entered	Alert: “Payment failed. Enter valid details”	Alert displayed	Pass
3	Successful purchase transaction	Alert: “Purchase successful!”	Purchase confirmed	Pass
4	Server error during checkout	Alert: “Transaction failed. Please try again.”	Alert displayed	Pass
5	No internet during transaction	Alert: “No network. Please try again.”	Alert displayed	Pass

CHAPTER 8

CONCLUSION

This Organic Farming System (OFS) is a key component in transforming conventional farming into a more intelligent, more sustainable, and farmer-friendly practice. By using modern technologies like digital platforms, crop management tools, certification tracking, and online marketplaces, the system helps farmers manage their farms better. They are able to communicate with customers directly and access real-time information about weather, soil health, and market trends. This decreases uncertainty in decision-making, boosts productivity, and improves the overall livelihood of farmers while ensuring that consumers receive healthy, chemical-free produce.

On the operational side, administrators and certification authorities benefit from centralized farmer data management. They enjoy streamlined certification approval processes and transparent monitoring of farming practices. By offering insightful information on resource utilisation, demand-supply imbalances, and crop production trends, the system facilitates better decision-making. In addition to enhancing confidence in the organic food supply chain, this advances the more general objectives of food security and sustainable agriculture.

The implementation shows how integrating MongoDB, Node.js, Express.js, and React.js, in addition to data visualisation tools and secure payment systems, can create a scalable, user-friendly, and reliable application. With features like farmer registration, crop listing, certification management, marketplace transactions, and weather alerts, the system connects technological innovation with practical agricultural needs.

In conclusion, the Organic Farming System is a significant step toward modernizing agriculture. It ensures greater transparency, empowers farmers, builds consumer trust, and promotes sustainable food production. As future developments in AI, IoT, and machine learning are included, These systems will grow further in predictive and adaptable, shaping the future of farming and contributing to a healthier, greener, and more sustainable world.

CHAPTER 9

FUTURE ENHANCEMENT

While the Organic Farming System (OFS) effectively tackles current challenges in sustainable farming, certification, and marketplace integration, there is a lot of room for further growth. Future improvements can focus on the following areas:

• Integration of Online Payment Systems: Adding secure digital payment gateways will allow farmers and buyers to complete transactions online. This ensures faster, safer, and cashless payments while promoting transparency in organic trade.

• Predictive Analytics for Crop Yield and Demand: By using machine learning algorithms, the system can review historical crop data, soil conditions, weather patterns, and market demand to predict crop yields and price changes. This will assist farmers in producing better decisions and cut losses.

• Mobile Application Development: Creating dedicated Android and iOS apps will improve accessibility. Farmers will have the opportunity to register crops, track certifications, receive weather alerts, and connect with buyers directly from their smartphones.

• IoT Integration for Smart Farming: The system can be improved by linking IoT-based sensors for temperature, soil moisture and pest detection. Real-time monitoring will help farmers use resources efficiently and maintain organic standards.

• Farmer-to-Farmer Knowledge Sharing Platform: Adding a discussion forum or chat feature will enable farmers to share experiences, techniques, and solutions for common farming issues, fostering a strong organic farming community.

• AI-Powered Advisory Services: Artificial Intelligence can offer personalized crop recommendations, pest management advice, and best practices for organic farming, adjusted to the farmer’s location and crop type.

• Blockchain for Certification and Traceability: Using blockchain technology will create secure, transparent, and tamper-proof certification management. Consumers will be able to trace products back to their source, building trust in organic produce.

APPENDIX A

BIBLIOGRAPHY

BOOKS REFERRED

[1] Lampkin, N. H., & Measures, M. (2001). Organic Farming. Old Pond Publishing.

[2] IFOAM – International Federation of Organic Agriculture Movements. (2016). Principles of Organic Agriculture. IFOAM.

[3] El-Hage Scialabba, N., & Hattam, C. (2002). Organic Agriculture, Environment and Food Security. FAO, Rome.

[4] Chodorow, K. (2019). MongoDB: The Definitive Guide: Powerful and Scalable Data Storage. O'Reilly Media.

[5] Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps. O'Reilly Media.

[6] Cantelon, M., Harter, M., Holowaychuk, T., & Rajlich, N. (2017). Node.js in Action. Manning Publications.

[7] Subramanian, V. (2019). Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node. Apress.

WEBSITES REFERRED

[1] https://www.fao.org/organicag/

[2] https://www.ifoam.bio/

[3] https://www.agriculture.com/

[4] https://react.dev/

[5] https://expressjs.com/

[6] https://www.mongodb.com/docs/

[7] https://nodejs.org/en/docs/

[8] https://stripe.com/docs/payments/

APPENDIX B

USER MANUAL (EMERGENCY RESPONSE SYSTEM)

1. OVERVIEW

This manual offers clear instructions for using, managing, and troubleshooting the Organic Farming System (OFS), which is built with the MERN stack (MongoDB, Express, React, Node.js). The system allows farmers to register, list crops, and manage organic certifications. Buyers can browse and purchase organic produce. Administrators can oversee user registrations, manage certifications, monitor transactions, and ensure compliance with organic standards. The system also includes weather alerts, marketplace features, and secure online transactions to promote sustainable agriculture.

2. SYSTEM REQUIREMENTS

Minimum Hardware

- **RAM**: 8 GB or higher
- **Storage**: 256 GB SSD (recommended)
- **Processor**: Intel Core i5 or equivalent
- **Display**: 15-inch or larger
- **Network**: Stable internet connection (Ethernet/Wi-Fi)

Software

- **Operating System**: Windows 10 / Ubuntu 20.04 LTS or later
- **Browser**: Chrome / Firefox (latest versions)
- **Node.js**: v16 or later with npm/yarn
- **MongoDB**: Local or Cloud instance (Atlas recommended)
- **IDE**: Visual Studio Code (recommended)

Environment / Configuration

Create an .env file with:

• MONGO_URI, MongoDB connection string

• JWT_SECRET, Secret key for authentication

• PAYMENT_GATEWAY_KEY, API key for online payments

• WEATHER_API_KEY, API key for weather updates

• PORT, Server port

3. INSTALLATION & SETUP (Developer / Admin)

Step 1: Clone Repository

git clone <repo-url> && cd <repo-folder>

- This command downloads the project from the repository (GitHub/GitLab/Bitbucket).
- The cd <repo-folder> navigates into the project's root directory where all backend and frontend code exists.

Step 2: Install Backend Dependencies

```
cd backend
npm install
```

- Navigate to the **backend folder**, which contains the Express.js server and APIs.
- Run npm install to download all required Node.js packages (express, mongoose, bcrypt, JWT, etc.) listed in the package.json.
- Without this, the backend server cannot function properly.

Step 3: Install Frontend Dependencies

```
cd ../frontend
npm install
```

- Navigate to the **frontend folder**, which contains the React.js client application.

- Run npm install to fetch all frontend libraries (React, Axios, Redux, Tailwind, etc.) required for UI rendering and API communication.

Step 4: Configure Environment Variables

- Create an .env file in both **backend** and **frontend** (if required).
- Add keys such as:
  - MONGO_URI → MongoDB database connection string.
  - JWT_SECRET → Used for secure authentication and token generation.
  - PAYMENT_GATEWAY_KEY → Secure online payments.
  - WEATHER_API_KEY → Fetch weather & alerts
- Environment variables ensure sensitive data is **kept secure** and not hardcoded in the source code.

Step 5: Start MongoDB (Local or Cloud)

- If using **local MongoDB**, ensure the MongoDB service is running (mongod).
- If using **MongoDB Atlas (cloud)**, verify that your connection string in .env (MONGO_URI) is correct.
- This is necessary for storing all data: users, responders, alerts, donations, and logs.

Step 6: Run Backend

```
cd backend
npm run dev
```

- Starts the **Express.js server** on a defined port (default: 5000).
- The backend handles **API requests, authentication, database operations, and alert management**.
- Running in dev mode usually enables hot reload using **nodemon**, so changes update automatically.

Step 7: Run Frontend

```
cd frontend
npm start
```

- Starts the **React.js development server** (default: http://localhost:3000).

- This launches the user interface (UI) where users, responders, and admins interact with the system.
- React will automatically connect with the backend APIs running on http://localhost:5000 (or configured proxy).

At this point, the full system is ready:

- Backend APIs → http://localhost:5000
- Frontend Application → http://localhost:3000

4. HOW TO USE (END USER)

4.1 Registration & Login

- **Sign Up**: Users (farmers/consumers) must register with name, email, phone number, and password.
- **Verification**: OTP or email verification ensures only genuine users join the platform.
- **Login**: Once verified, login generates a secure JWT session token.

4.2 Browsing & Ordering products

- **Product Catalog**: Consumers can view available organic products (vegetables, fruits, dairy, etc.).
- **Search & Filter**: Products can be filtered by category, price, or farmer.
- **Add to Cart**: Items can be added to the shopping cart for easy checkout.
- **Place Order**: Finalize purchase and proceed to payment

4.3 Payment & Order Confirmation

- **Payment Options**: UPI, Debit/Credit Card, Net Banking.
- **Order Confirmation**: After payment, an invoice and order ID are generated.
- **Notifications:** Users receive email/SMS updates on order status.

4.4 Tracking Orders

- **Order Status**: Users can track whether their order is pending, packed, shipped, or delivered.
- **Delivery Updates**: Real-time notifications are sent at each stage.

4.5 Logging Out

- Always log out to clear session and prevent unauthorized access.

5. FARMER FEATURES

5.1 Farmer Registration

- Farmers must provide personal details, farm location, and organic certification documents.
- Admin Verification is required before products can be listed.

5.2 Product Management

- Farmers can add, edit, or delete products with details like price, stock, and description.
- Upload product images for better visibility.

5.3 Order Handling

- Farmers receive notifications when new orders are placed.
- Options: Accept, Pack, or Mark as Delivered.

5.4 Earnings & Reports

- Farmers can view total sales, earnings reports, and customer feedback.

6. ADMIN FEATURES

6.1 Admin Login

- Admin accounts are pre-configured with higher access privileges.

6.2 Dashboard Functions

- **Approve Farmers**: Verify farmer details and documents.
- **Manage Products**: Monitor categories, approve/reject listings.
- **Track Orders**: Oversee delivery status and dispute resolution.
- **Payment Oversight:** Ensure smooth functioning of online transactions.
- **Reports**: Generate sales, user activity, and system performance reports.

6.3 Notifications & Support

- Send platform-wide announcements.
- Monitor customer queries and farmer issues.

7. OUTPUT & RESPONSE HANDLING

- Orders and payments are securely logged with timestamps and user details.
- Farmers and consumers receive **instant notifications**.
- Admins can monitor full platform activity in real-time.

8. TROUBLESHOOTING

- **Problem: Product not showing**
  - Check internet connection.
  - Ensure farmer account is admin-verified.
- **Problem: Payment failed**
  - Verify payment gateway API.
  - Cross-check transaction logs.
- **Problem: Order not updating**
  - Ensure backend APIs are running.
  - Refresh order dashboard.
- **Problem: Email/SMS not received**
  - Verify MAIL_SERVICE_KEY configuration.

9. SUPPORT & CONTACT

For assistance:

- **Email**: support@organicfarmizean.com
- **Phone**: +91-8431457057
- A dedicated support team is available for bug fixing, deployment help, and technical guidance.